

DECADE

Université de la Manouba  
Ecole Nationale des Sciences de l'Informatique



## Rapport du Projet de Fin d'Etudes

---

**Sujet : Conception et réalisation d'un  
module intelligent de validation des données  
produits sous SAP Commerce Cloud**

---

*Réalisé par :*

M<sup>lle</sup>. Imen BOUSSETTA

*Encadrante Académique :*

Mme. Hela BOUKEF

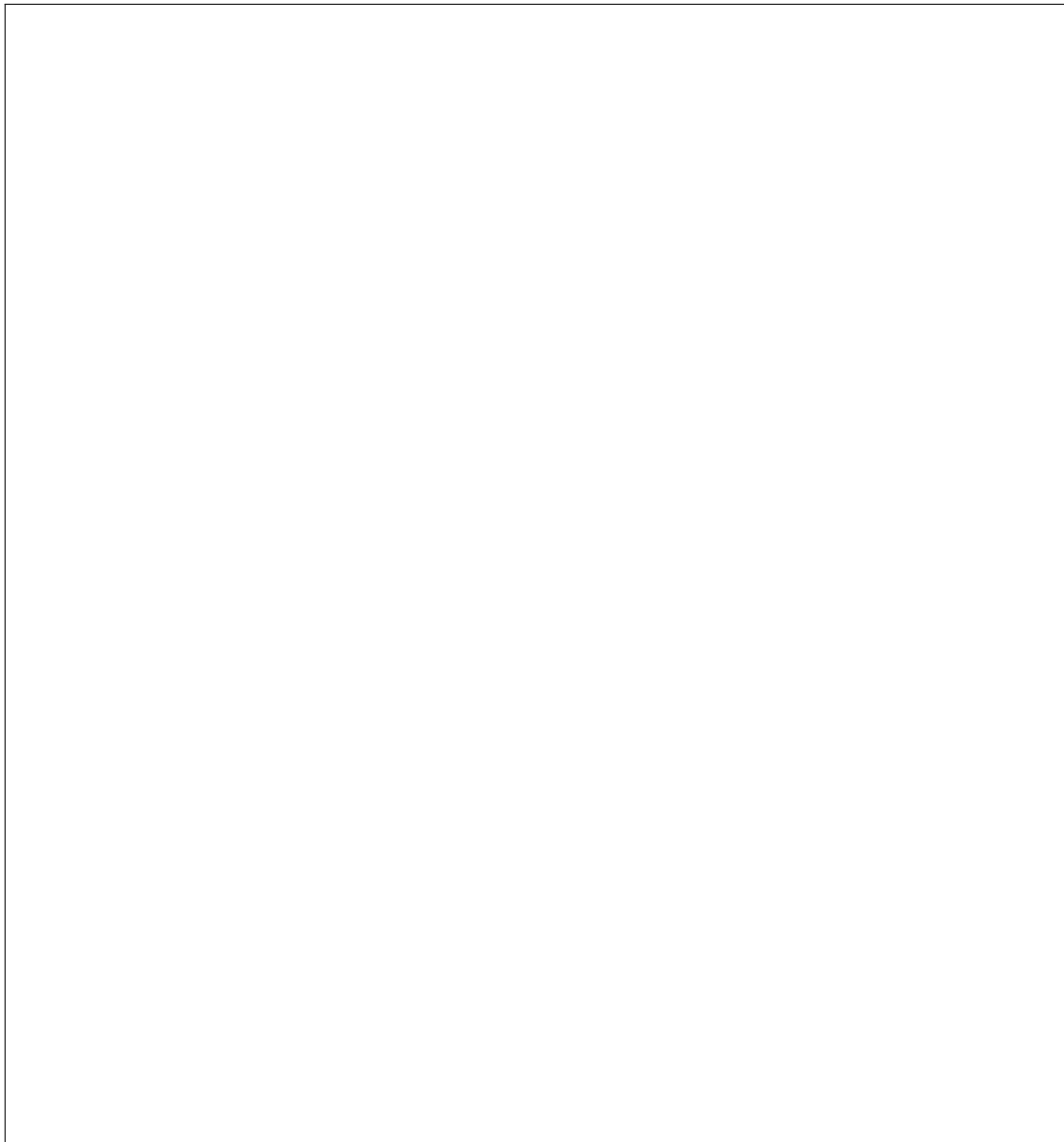
*Encadrant Professionnel :*

M. Aymen ELLOUZE

---

Année universitaire : 2025 /2026

# Appréciations et signature de l'encadrant

A large, empty rectangular box with a thin black border, intended for the supervisor's appreciations and signature.

## Résumé

Ce projet s'inscrit dans le cadre de notre projet de fin d'études à l'École Nationale des Sciences de l'Informatique, en vue de l'obtention du diplôme d'ingénieur en informatique. Il a été réalisé au sein de DECADE sur une durée de six mois.

L'objectif principal du projet est de concevoir une solution intelligente d'enrichissement et de validation des données produits intégrée à SAP Commerce pour ETANCO/SIMPSON. Cette solution permet d'automatiser la génération des descriptions produits, la traduction des contenus ainsi que la détection des anomalies et incohérences dans les données avant leur publication sur les plateformes e-commerce.

**Mots-clés :** SAP Commerce, Intelligence artificielle, NLP, Enrichissement produit, Validation des données, E-commerce.

---

## Abstract

This project is part of our end-of-study project at the National School of Computer Science, leading to the engineering degree in computer science. It was carried out at DECADE over a period of six months.

The main objective of the project is to design an intelligent product enrichment and validation solution integrated into SAP Commerce for ETANCO/SIMPSON. The solution automates product description generation, content translation, and the detection of anomalies and inconsistencies in product data before publication on e-commerce platforms.

**Keywords :** SAP Commerce, Artificial Intelligence, NLP, Product Enrichment, Data Validation, E-commerce.

---

## Résumé Arabe

# Remerciements

texte

texte

texte

# Table des matières

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Introduction Générale</b>                                           | <b>1</b>  |
| <b>1 Contexte général</b>                                              | <b>4</b>  |
| 1.1 Présentation de l'organisme d'accueil . . . . .                    | 4         |
| 1.1.1 Présentation de l'entreprise : DECADE . . . . .                  | 4         |
| 1.1.2 Types de projets . . . . .                                       | 5         |
| 1.1.3 Présentation du client : ETANCO . . . . .                        | 5         |
| 1.2 Concepts de base . . . . .                                         | 6         |
| 1.2.1 Le commerce électronique . . . . .                               | 6         |
| 1.2.2 Les systèmes intervenant dans le commerce électronique . . . . . | 7         |
| 1.2.3 Cycle de vie des données d'un produit . . . . .                  | 8         |
| 1.3 Cadre général du projet . . . . .                                  | 9         |
| 1.3.1 Contexte . . . . .                                               | 9         |
| 1.3.2 Étude de l'existant . . . . .                                    | 9         |
| 1.4 Problématique et solution proposée . . . . .                       | 9         |
| 1.4.1 Problématique . . . . .                                          | 9         |
| 1.4.2 Solution proposée . . . . .                                      | 10        |
| 1.5 Méthodologie de travail adoptée . . . . .                          | 10        |
| 1.5.1 Éléments de la méthodologie Scrum . . . . .                      | 12        |
| 1.5.2 Rôles au sein de l'équipe Scrum . . . . .                        | 12        |
| 1.5.3 Mécanisme de feedback . . . . .                                  | 12        |
| 1.6 Conclusion du chapitre . . . . .                                   | 14        |
| <b>2 Conception, Modélisation et Architecture</b>                      | <b>15</b> |
| 2.1 Introduction du chapitre . . . . .                                 | 15        |
| 2.2 Spécification des besoins . . . . .                                | 15        |
| 2.2.1 Identification des acteurs . . . . .                             | 15        |
| 2.2.2 Recueil des besoins fonctionnels . . . . .                       | 16        |
| 2.2.3 Recueil des besoins non fonctionnels . . . . .                   | 17        |

|          |                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------|-----------|
| 2.3      | Modélisation des besoins . . . . .                                                           | 18        |
| 2.3.1    | Diagramme de cas d'utilisation . . . . .                                                     | 19        |
| 2.3.2    | Diagramme de séquence . . . . .                                                              | 20        |
| 2.3.3    | Diagramme d'activités . . . . .                                                              | 20        |
| 2.4      | Architecture globale . . . . .                                                               | 22        |
| 2.4.1    | Vue générale de l'architecture . . . . .                                                     | 22        |
| 2.4.2    | Description détaillée des composants . . . . .                                               | 24        |
| 2.4.3    | Communication entre SAP Commerce et le microservice . . . . .                                | 25        |
| 2.4.4    | Flux global de traitement . . . . .                                                          | 26        |
| 2.4.5    | Justification des choix architecturaux . . . . .                                             | 26        |
| 2.4.6    | Justification des choix technologiques . . . . .                                             | 27        |
| 2.5      | Conclusion du chapitre . . . . .                                                             | 27        |
| <b>3</b> | <b>Module d'enrichissement et de validation des produits</b>                                 | <b>28</b> |
| 3.1      | Introduction . . . . .                                                                       | 28        |
| 3.2      | Conception du microservice . . . . .                                                         | 28        |
| 3.2.1    | Déroulement du traitement . . . . .                                                          | 29        |
| 3.2.2    | Choix technologiques . . . . .                                                               | 29        |
| 3.3      | Module d'enrichissement intelligent . . . . .                                                | 31        |
| 3.3.1    | Génération automatique des descriptions . . . . .                                            | 31        |
| 3.3.2    | Module de traduction multilingue . . . . .                                                   | 34        |
| 3.3.3    | Structure de la réponse du service d'enrichissement . . . . .                                | 37        |
| 3.3.4    | Module de validation : contrôle de qualité automatique . . . . .                             | 40        |
| 3.3.5    | Standardisation de la réponse de validation . . . . .                                        | 43        |
| 3.4      | Conclusion du chapitre . . . . .                                                             | 45        |
| <b>4</b> | <b>Intégration dans SAP Commerce Cloud</b>                                                   | <b>46</b> |
| 4.1      | Introduction . . . . .                                                                       | 46        |
| 4.2      | Architecture logicielle de SAP Commerce Cloud . . . . .                                      | 46        |
| 4.2.1    | Fondamentaux de SAP Commerce Cloud . . . . .                                                 | 47        |
| 4.2.2    | Les couches de SAP Commerce . . . . .                                                        | 48        |
| 4.3      | Communication entre SAP Commerce Cloud et le module d'intelligence<br>artificielle . . . . . | 50        |
| 4.3.1    | Principe de communication REST . . . . .                                                     | 50        |
| 4.3.2    | API d'enrichissement et API de validation . . . . .                                          | 51        |
| 4.3.3    | Flux global de communication . . . . .                                                       | 51        |
| 4.4      | Extension du modèle de données SAP Commerce . . . . .                                        | 52        |

|                                   |                                                                        |           |
|-----------------------------------|------------------------------------------------------------------------|-----------|
| 4.4.1                             | Adaptation du modèle produit . . . . .                                 | 52        |
| 4.4.2                             | Gestion des résultats de validation . . . . .                          | 53        |
| 4.4.3                             | Gestion des anomalies détectées . . . . .                              | 53        |
| 4.4.4                             | Relations entre les éléments du modèle . . . . .                       | 53        |
| 4.5                               | Interfaces Backoffice pour l'enrichissement et la validation . . . . . | 54        |
| 4.5.1                             | Vue d'ensemble du Backoffice SAP Commerce . . . . .                    | 54        |
| 4.5.2                             | Interface d'enrichissement . . . . .                                   | 55        |
| 4.5.3                             | Interface de validation . . . . .                                      | 57        |
| 4.6                               | Workflows métier et automatisation . . . . .                           | 60        |
| 4.6.1                             | Workflows dans SAP Commerce . . . . .                                  | 60        |
| 4.6.2                             | Workflow d'enrichissement . . . . .                                    | 61        |
| 4.6.3                             | Workflow de validation . . . . .                                       | 63        |
| 4.7                               | Flux complet d'exécution et orchestration . . . . .                    | 64        |
| 4.7.1                             | Scénario concret : enrichissement d'un produit . . . . .               | 64        |
| 4.7.2                             | Étapes techniques du flux complet . . . . .                            | 65        |
| 4.8                               | Système de notification par courrier électronique . . . . .            | 71        |
| 4.8.1                             | Moteur d'email dans SAP Commerce . . . . .                             | 71        |
| 4.8.2                             | Moteurs de template Velocity . . . . .                                 | 71        |
| 4.8.3                             | Configuration SMTP et Mailtrap . . . . .                               | 73        |
| 4.8.4                             | Processus complet de notification . . . . .                            | 73        |
| 4.9                               | Conclusion du chapitre . . . . .                                       | 74        |
| <b>Conclusion et perspectives</b> |                                                                        | <b>77</b> |
| <b>A Annexe 1</b>                 |                                                                        | <b>78</b> |
| <b>Bibliographie</b>              |                                                                        | <b>79</b> |
| <b>Netographie</b>                |                                                                        | <b>80</b> |



# Liste des acronymes

|             |                                    |
|-------------|------------------------------------|
| <b>AI</b>   | Artificial Intelligence            |
| <b>API</b>  | Application Programming Interface  |
| <b>B2B</b>  | Business-to-Business               |
| <b>B2C</b>  | Business-to-Consumer               |
| <b>BLEU</b> | Bilingual Evaluation Understudy    |
| <b>CRM</b>  | Customer Relationship Management   |
| <b>DAO</b>  | Data Access Object                 |
| <b>DTO</b>  | Data Transfer Object               |
| <b>ERP</b>  | Enterprise Resource Planning       |
| <b>GPU</b>  | Graphics Processing Unit           |
| <b>HTML</b> | HyperText Markup Language          |
| <b>HTTP</b> | HyperText Transfer Protocol        |
| <b>IA</b>   | Intelligence Artificielle          |
| <b>JSON</b> | JavaScript Object Notation         |
| <b>LLM</b>  | Large Language Model               |
| <b>LoRA</b> | Low-Rank Adaptation                |
| <b>NLP</b>  | Natural Language Processing        |
| <b>NLLB</b> | No Language Left Behind            |
| <b>ORM</b>  | Object-Relational Mapping          |
| <b>PEFT</b> | Parameter-Efficient Fine-Tuning    |
| <b>PIM</b>  | Product Information Management     |
| <b>REST</b> | Representational State Transfer    |
| <b>SMTP</b> | Simple Mail Transfer Protocol      |
| <b>TAL</b>  | Traitement Automatique des Langues |

|            |                            |
|------------|----------------------------|
| <b>UML</b> | Unified Modeling Language  |
| <b>URL</b> | Uniform Resource Locator   |
| <b>XML</b> | Extensible Markup Language |

# Introduction Générale

Le commerce en ligne est devenu un canal de vente indispensable pour la plupart des entreprises. Sur une plateforme e-commerce, la qualité des informations présentées aux clients est un facteur déterminant pour réussir une vente. Des informations complètes et précises sur les produits ne constituent pas seulement un support technique, elles rassurent l'acheteur et diminuent les risques de retours. Pour les entreprises qui gèrent des milliers de références, comme c'est le cas dans le milieu industriel, les données relatives aux produits sont donc un actif stratégique qu'il faut savoir valoriser.

Pourtant, la gestion d'un catalogue volumineux pose souvent des problèmes concrets aux équipes métier. Chez de nombreux distributeurs ou fabricants, l'enrichissement des informations reste une tâche essentiellement manuelle et répétitive. Les personnes chargées de l'administration du catalogue doivent souvent traduire des textes, rédiger des descriptions techniques et vérifier la cohérence de nombreux attributs des produits. Ce processus est long, coûteux et peut facilement laisser passer des erreurs humaines. Ces incohérences finissent par nuire à la crédibilité du site et à l'expérience de l'utilisateur final.

L'évolution récente des technologies d'intelligence artificielle offre de nouvelles pistes pour simplifier ce travail. Les modèles de traitement du langage naturel, ou NLP, ainsi que les modèles de langage de grande taille (LLM), sont désormais capables de comprendre et de générer du texte avec une précision remarquable. Ces outils permettent d'automatiser des tâches qui, jusqu'à présent, nécessitaient une intervention humaine constante. En intégrant ces modèles dans les processus de gestion, il devient possible de traiter les données des produits plus rapidement tout en maintenant un niveau de qualité élevé.

C'est dans ce contexte que s'inscrit mon projet de fin d'études au sein de l'entreprise DECADE [1]. Spécialisée dans les solutions de commerce électronique, DECADE accompagne ses clients dans la mise en place de plateformes performantes. Mon stage s'est concentré sur un besoin spécifique exprimé par ETANCO [2], une filiale du groupe Simpson Strong-Tie, qui fabrique et distribue des systèmes de fixation. Avec un catalogue technique très riche, l'entreprise cherchait un moyen efficace d'automatiser l'enrichissement et la vérification du contenu associé aux produits au sein de son outil de gestion, SAP

Commerce Cloud [6].

Le travail que j’ai réalisé consiste à concevoir et développer un module intelligent capable d’assister les équipes d’ETANCO. L’idée principale est de proposer un système qui ne se contente pas de stocker des informations, mais qui aide activement à les améliorer. Ce module permet notamment de générer automatiquement des descriptions des produits détaillées à partir de simples caractéristiques techniques, de traduire les informations relatives aux produits en plusieurs langues et de détecter les anomalies ou les éléments manquants avant la mise en ligne.

Pour répondre à ces besoins, j’ai mis en place une architecture qui fait le pont entre le monde du e-commerce et celui de l’intelligence artificielle. D’un côté, le système utilise Java et le framework Spring [?], qui sont les bases technologiques de SAP Commerce Cloud. De l’autre, j’ai développé une interface de services sous Python [14] avec le framework FastAPI [13] pour exploiter les modèles d’intelligence artificielle. Cette séparation permet d’utiliser le meilleur de chaque technologie : la robustesse de SAP pour la gestion commerciale et la flexibilité de Python pour les traitements d’intelligence artificielle.

L’un des apports majeurs de cette solution est la mise en place d’un score de qualité pour chaque présentation de produit. Ce score permet aux responsables de voir immédiatement quelles données des produits sont prêtes à être publiées et lesquelles demandent une correction. En suggérant des modifications automatiques et en signalant les incohérences dans les caractéristiques des produits, l’outil devient un véritable assistant qui réduit considérablement le temps passé sur des tâches à faible valeur ajoutée. L’objectif final est de permettre aux équipes de se concentrer sur la stratégie plutôt que sur la correction manuelle de fichiers.

Ce rapport présente l’intégralité des étapes de ce projet, de l’analyse initiale à la validation des résultats en production.

Le chapitre 1 expose le contexte général du projet : qui est DECADE, qui est ETANCO, quels sont les enjeux du e-commerce et de la qualité des données, et quelle est précisément la problématique que nous cherchons à résoudre.

Le chapitre 2 détaille l’analyse des besoins et la conception de l’architecture générale du système. Nous y présentons les besoins fonctionnels et non-fonctionnels, les acteurs impliqués, et la stratégie architecturale retenue (microservices découplés, API REST, séparation SAP/IA).

Le chapitre 3 constitue le cœur technique du projet. Il détaille le module d’enrichissement et de validation des produits alimenté par l’intelligence artificielle. Nous y expliquons les pipelines d’enrichissement (génération de descriptions, traduction multilingue), les pipelines de validation (règles métier, analyse linguistique et sémantique),

le calcul du score qualité, et finalement l'intégration de cette intelligence dans SAP Commerce Cloud (services Java, workflows, extensions du Backoffice).

Ces trois chapitres forment un tout cohérent : problématique → architecture → solution.

# Chapitre 1

## Contexte général

### Introduction du chapitre

Ce chapitre présente le contexte général du projet ainsi que son cadre global. Il introduit d'abord l'organisme d'accueil DECADE Tunisie ainsi que son client ETANCO, avant de détailler les concepts fondamentaux liés à la gestion des informations des produits.

Nous analysons ensuite le contexte du projet et l'étude de l'existant afin de mettre en lumière la problématique identifiée. Enfin, nous présentons la solution proposée ainsi que la méthodologie Agile Scrum adoptée pour structurer le déroulement de ce travail.

### 1.1 Présentation de l'organisme d'accueil

Dans cette section, nous présentons l'organisme d'accueil, le client concerné par le projet ainsi que les principaux domaines dans lesquels ils exercent leurs activités.

#### 1.1.1 Présentation de l'entreprise : DECADE

DECADE [1] est une Entreprise de Services du Numérique (ESN) spécialisée dans le conseil et la réalisation de solutions e-commerce et de transformation digitale. Depuis sa création en 1988, elle s'est imposée comme un partenaire stratégique pour les entreprises souhaitant titulariser leur activité commerciale en ligne.

L'expertise de DECADE repose sur une maîtrise approfondie des outils de vente en ligne, notamment SAP Commerce Cloud et sur une forte capacité à intégrer des problématiques complexes de gestion des informations des produits. L'entreprise accompagne ses clients tout au long du cycle de vie de leurs projets, de la conception

logicielle à l'optimisation continue, en mettant l'accent sur la qualité des contenus et l'efficacité des processus métier.

Cette spécialisation dans le commerce électronique à forte valeur ajoutée constitue le cadre idéal pour ce projet de fin d'études, qui s'articule autour de l'amélioration du parcours d'enrichissement des informations des produits pour un client industriel majeur. L'identité visuelle de l'entreprise est illustrée par la Figure 1.1.

## DECADE

FIGURE 1.1 – Logo de DECADE

### 1.1.2 Types de projets

DECADE accompagne ses clients à travers des solutions adaptées à leurs objectifs stratégiques. Les principaux axes d'intervention de l'entreprise sont :

- **Solutions e-commerce** : conception et développement de plateformes de vente robustes et évolutives.
- **Gestion des informations des produits (PIM/PXM)** : mise en place de référentiels uniques pour centraliser et enrichir les données des produits.
- **Expérience utilisateur (UX/UI)** : création de parcours d'achat ergonomiques et performants adaptés aux besoins des clients.
- **Marketplaces** : déploiement de solutions permettant d'étendre l'offre commerciale via des écosystèmes multipartenaires.
- **Gestion de la relation client (CRM)** : intégration d'outils de suivi et de fidélisation pour une meilleure connaissance du client.
- **Architectures API-first et Cloud** : mise en œuvre de systèmes modulaires et flexibles facilitant l'interopérabilité entre les outils.

### 1.1.3 Présentation du client : ETANCO

Dans le cadre de ce projet, DECADE intervient pour le compte d'ETANCO [2], une filiale du groupe international Simpson Strong-Tie. ETANCO est un acteur majeur en Europe dans la conception, la fabrication et la distribution de solutions de fixation et

d'assemblage destinées à l'enveloppe du bâtiment. L'entreprise intervient dans plusieurs domaines, notamment la toiture, la façade, le bardage et l'étanchéité et s'adresse principalement aux professionnels du bâtiment.

Afin de répondre aux besoins de ses clients, ETANCO commercialise un catalogue comprenant plusieurs dizaines de milliers de références. Chaque produit est associé à de nombreuses informations, telles que des caractéristiques techniques, des descriptions, des documents réglementaires, des médias, des classifications ainsi que des traductions destinées aux différents marchés. La qualité, la cohérence et la disponibilité de ces informations sont essentielles pour assurer une présentation fiable des produits sur les différents canaux de diffusion de l'entreprise. La Figure 1.2 présente l'identité visuelle du groupe.



FIGURE 1.2 – Logo d'ETANCO

## 1.2 Concepts de base

Cette section introduit les notions fondamentales nécessaires à la compréhension de la circulation des informations au sein d'une interface de vente moderne.

### 1.2.1 Le commerce électronique

Le commerce électronique, ou e-commerce, désigne l'échange de biens, de services ou d'informations par l'intermédiaire de réseaux informatiques, principalement Internet [3]. Il ne se limite pas à l'acte d'achat, mais englobe l'ensemble du parcours client numérique.

On distingue généralement deux modèles principaux selon la nature des acteurs impliqués :

- **B2C (Business-to-Consumer)** : les entreprises vendent directement aux particuliers. Le processus de décision est souvent court et émotionnel.
- **B2B (Business-to-Business)** : les échanges s'effectuent entre deux entités morales. Ce modèle, propre au contexte d'ETANCO, se caractérise par des catalogues complexes et des processus d'achat rationnels.

Le tableau 1.1 synthétise les disparités majeures entre ces deux approches.



| Caractéristique   | E-commerce B2C            | E-commerce B2B                        |
|-------------------|---------------------------|---------------------------------------|
| Client cible      | Particulier (Individu)    | Professionnel (Entreprise)            |
| Processus d'achat | Rapide et impulsif        | Long, structuré et validé             |
| Volume / Prix     | Unitaire / Prix fixe      | Quantités importantes / Prix négociés |
| Relation client   | Ponctuelle et court terme | Partenariale et pérenne               |
| Catalogue         | Standardisé               | Complexe et technique                 |

TABLE 1.1 – Comparaison entre les modèles e-commerce B2B et B2C

Cette comparaison met en évidence les différences fondamentales entre les deux approches e-commerce. Dans le contexte d'ETANCO, le modèle B2B prédomine : les clients sont des professionnels du bâtiment qui nécessitent des informations techniques précises, des catalogs complets et des relations commerciales structurées. Cette spécificité impose des exigences particulières en termes de qualité et d'exactitude des données des produits. Une plateforme e-commerce s'appuie structurellement sur deux composantes :

- **Front Office (Storefront)** : l'interface publique destinée aux utilisateurs pour la navigation et l'acquisition de produits.
- **Back Office** : l'interface d'administration permettant de gérer les catalogues, les commandes et les clients.

### 1.2.2 Les systèmes intervenant dans le commerce électronique

La gestion efficace d'une boutique en ligne repose sur l'interopérabilité de plusieurs logiciels métiers spécialisés :

- **ERP (Enterprise Resource Planning)** : colonne vertébrale de l'entreprise, il centralise les données opérationnelles telles que les stocks, les tarifs et la comptabilité [4].
- **PIM (Product Information Management)** : référentiel unique dédié à la centralisation, l'enrichissement et la traduction des caractéristiques techniques et commerciales des produits [5].
- **SAP Commerce Cloud** : plateforme cloud omnicanale qui orchestre l'expérience client et le pilotage des processus de vente complexes [6].
- **CRM (Customer Relationship Management)** : outil de gestion de la relation client permettant de suivre les interactions et d'optimiser les stratégies de fidélisation [7].

### 1.2.3 Cycle de vie des données d'un produit

Pour garantir la pertinence de l'offre en ligne, les informations des produits suivent un cycle de vie structuré à travers les différents systèmes. L'architecture technique de ce flux est illustrée dans la Figure 1.3.

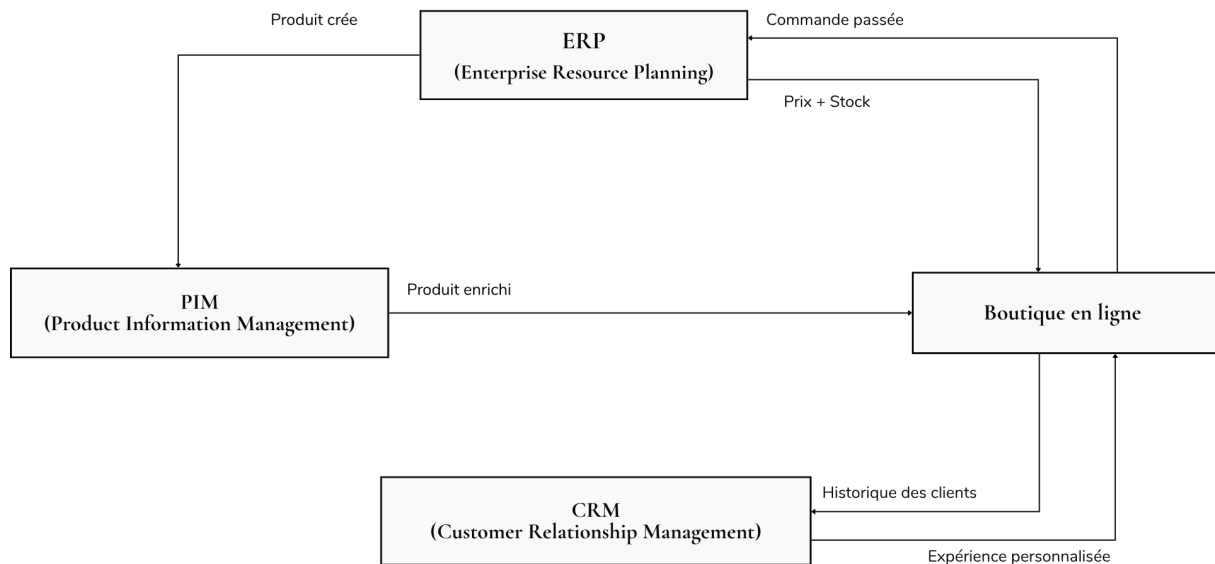


FIGURE 1.3 – Flux de circulation des informations des produits dans l'écosystème e-commerce

Le flux commence par la **création** de la référence de base dans l'ERP. Le produit est ensuite transmis au PIM pour son **enrichissement** (attributs des produits techniques, médias, traductions). Une fois validées, ces données des produits sont **publiées** vers SAP Commerce Cloud pour être **consultées** par les clients sur le Storefront.

Lors de la transaction, SAP Commerce synchronise en temps réel les prix et les stocks avec l'ERP pour finaliser l'achat. Enfin, chaque interaction alimente le CRM pour affiner la connaissance client. La collaboration entre ces systèmes est cruciale : l'ERP assure la fiabilité opérationnelle, le PIM garantit la richesse du contenu, SAP Commerce gère la vente et le CRM valorise les données du client.

## 1.3 Cadre général du projet

### 1.3.1 Contexte

Dans le cadre de son activité e-commerce, ETANCO gère un catalogue technique très vaste comprenant des fixations, des accessoires ainsi que des systèmes de toiture. Chaque produit est décrit à travers un ensemble riche d'attributs des produits, souvent structurés et dépendants de sa catégorie.

Les données des produits sont principalement saisies et enrichies dans le backoffice SAP Commerce. Elles sont également complétées via des imports ou des mises à jour manuelles réalisées par les équipes métier.

Cependant, malgré les outils existants, certaines informations techniques et commerciales des produits peuvent rester incomplètes ou contenir des erreurs de saisie, ce qui rend les opérations de contrôle complexes avant la mise en ligne sur les plateformes e-commerce.

### 1.3.2 Étude de l'existant

Actuellement, la publication d'un produit sur le site e-commerce repose sur un processus largement manuel. Lorsqu'un produit est créé ou modifié, plusieurs vérifications doivent être réalisées, notamment le contrôle des noms, des descriptions des produits, des attributs des produits ainsi que des images associées.

Cette phase se termine par une validation effectuée par le *Product Detail Manager* dans le workflow d'enrichissement intégré à SAP Commerce. Bien que cette étape soit essentielle pour garantir la qualité des informations publiées, elle dépend fortement de l'intervention humaine.

En raison du volume important d'informations à traiter, certaines validations peuvent être partielles. Cela peut entraîner la présence d'erreurs ou d'incohérences dans les informations des produits publiées sur le Storefront.

## 1.4 Problématique et solution proposée

### 1.4.1 Problématique

Dans un contexte e-commerce, les clients s'appuient sur les caractéristiques des produits pour prendre leurs décisions d'achat. Toute erreur ou information manquante peut donc avoir un impact direct sur la confiance des clients sur les ventes.

Or, le processus actuel de validation manuelle présente plusieurs limites. Il peut laisser passer certaines anomalies telles que des attributs des produits manquants, des incohérences entre les données ou des problèmes de catégorisation.

Ces problèmes de qualité réduisent la fiabilité des informations publiées. Il devient ainsi nécessaire de fiabiliser le processus d'enrichissement et de validation des données des produits.

### 1.4.2 Solution proposée

Afin de répondre à cette problématique, la solution proposée consiste à intégrer une couche d'analyse intelligente dans le workflow d'enrichissement de SAP Commerce.

La solution repose sur un service externe accessible via des API REST permettant l'échange des données nécessaires à l'analyse et à la validation. Cette approche vise à renforcer les contrôles de qualité tout en conservant la validation humaine. Elle permet d'automatiser certaines tâches d'enrichissement, telles que la génération de descriptions des produits et la traduction des contenus ainsi que la normalisation des formats de données.

L'utilisateur conserve un rôle central dans le processus, puisqu'il peut valider ou refuser les suggestions proposées avant leur publication officielle sur le site.

Cette solution contribue à rendre le processus d'enrichissement plus efficace et plus fiable, tout en améliorant la qualité des informations techniques et commerciales des produits publiées en ligne.

## 1.5 Méthodologie de travail adoptée

Dans le cadre de ce projet, nous avons adopté la méthodologie Agile Scrum [8], largement utilisée dans le développement logiciel pour sa flexibilité et sa capacité à s'adapter aux besoins évolutifs du projet. Cette approche repose sur un développement itératif et incrémental, permettant de livrer progressivement des fonctionnalités tout en intégrant des retours réguliers des parties prenantes afin d'améliorer continuellement le produit final. Le fonctionnement général de cette méthodologie est présenté dans la Figure 1.4.

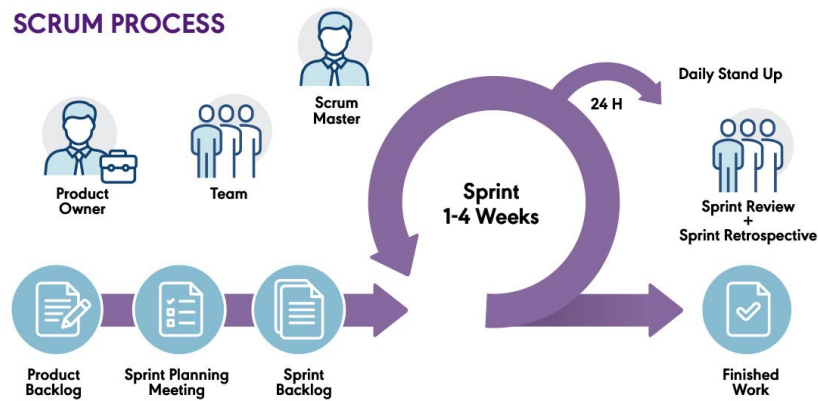


FIGURE 1.4 – Principe de la méthodologie Scrum

Ce diagramme illustre l'interaction cyclique entre les différents éléments Scrum : la planification initiale, l'exécution des Sprints, les réunions d'examen et les phases de rétrospective. Cette structure permet une amélioration continue tout en assurant une livraison régulière de fonctionnalités opérationnelles.

### 1.5.1 Éléments de la méthodologie Scrum

La méthodologie Scrum repose sur un ensemble d'éléments essentiels qui structurent le processus de développement :

- **Product Backlog** : il s'agit d'une liste priorisée regroupant l'ensemble des fonctionnalités, améliorations et corrections à réaliser sur le produit. Sa gestion est assurée par le Product Owner.
- **Sprint Backlog** : ensemble des tâches sélectionnées à partir du Product Backlog et que l'équipe s'engage à réaliser durant un Sprint.
- **Sprint** : période de développement courte et fixe, généralement de deux à quatre semaines, durant laquelle un incrément fonctionnel du produit est développé.
- **Daily Scrum** : réunion quotidienne de synchronisation permettant de suivre l'avancement, d'identifier les blocages et d'ajuster les priorités.
- **Sprint Review** : réunion organisée à la fin de chaque Sprint afin de présenter les fonctionnalités développées et de recueillir les retours des parties prenantes.
- **Sprint Retrospective** : réunion d'analyse permettant à l'équipe d'identifier les points positifs, les difficultés rencontrées et les axes d'amélioration pour les prochains Sprints.

### 1.5.2 Rôles au sein de l'équipe Scrum

Dans le cadre de ce projet, les rôles Scrum ont été répartis de la manière suivante :

- **Chef de projet technique** : Monsieur Aymen Ellouze, responsable de la coordination technique du projet, de la répartition des tâches et du suivi global de l'avancement.
- **Équipe de développement** : chargée de la conception, de l'implémentation, des tests et de la livraison des différentes fonctionnalités du système.

### 1.5.3 Mécanisme de feedback

Le mécanisme de feedback constitue un élément clé dans l'amélioration continue du projet. Il repose sur des échanges réguliers entre les membres de l'équipe et le chef de projet.

Ainsi, des réunions quotidiennes de type Daily Scrum sont organisées afin de partager l'état d'avancement et d'identifier rapidement les éventuels blocages. En complément, des réunions de suivi sont tenues toutes les une à deux semaines afin de présenter l'avancement global du projet et de valider les fonctionnalités développées.

Ces interactions régulières permettent d'assurer une meilleure adaptation aux besoins du projet et de garantir un alignement constant entre les développements réalisés et les objectifs fixés.

## 1.6 Conclusion du chapitre

Dans ce chapitre, nous avons présenté le cadre général du projet ainsi que l'environnement dans lequel il s'inscrit. Nous avons introduit l'organisme d'accueil DECADE Tunisie et mis en évidence ses activités dans le domaine du commerce en ligne.

Nous avons également analysé le contexte du projet ainsi que le processus existant de gestion des informations des produits, ce qui nous a permis d'identifier les limites actuelles et de définir la problématique.

Enfin, nous avons présenté la solution proposée ainsi que la méthodologie de travail adoptée, basée sur l'approche Agile Scrum, qui structure efficacement le déroulement du projet et garantit une progression itérative et contrôlée.



# Chapitre 2

## Conception, Modélisation et Architecture

### 2.1 Introduction du chapitre

Ce chapitre traduit les enjeux métier en spécifications techniques et présente l'architecture du système. Nous formalisons d'abord les besoins fonctionnels et non fonctionnels, puis proposons une modélisation UML des interactions. Enfin, nous décrivons l'architecture basée sur la séparation entre SAP Commerce et le microservice d'intelligence artificielle.

### 2.2 Spécification des besoins

Dans cette section, nous formalisons les exigences du système en distinguant les besoins fonctionnels, qui décrivent les services attendus et les besoins non fonctionnels, qui définissent les contraintes de qualité et de performance. Cette étape traduit les attentes métier en spécifications exploitables pour la conception.

#### 2.2.1 Identification des acteurs

Tout système interactif repose sur une ou plusieurs catégories d'utilisateurs. Identifier précisément ces acteurs définit les fonctionnalités attendues.

#### **Le responsable de l'enrichissement du produit**

L'utilisateur principal du système est le responsable de l'enrichissement du produit. Il intervient dans la gestion, l'amélioration et la validation des fiches des produits au sein de

SAP Commerce. Il crée, modifie ou valide régulièrement les données des produits avant leur publication.

Son rôle consiste à garantir la qualité et la complétude des informations des produits. Le système proposé l'assiste en générant automatiquement des enrichissements et en signalant les anomalies détectées. Cependant, le responsable de l'enrichissement du produit conserve l'autorité finale : il examine les propositions du système et approuve ou rejette chaque suggestion en fonction de son jugement professionnel.

## 2.2.2 Recueil des besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le responsable de l'enrichissement du produit doit pouvoir accomplir au travers du système. Ils structurent l'interaction entre l'utilisateur et la solution.

### Enrichissement des contenus des produits

Le responsable de l'enrichissement du produit doit pouvoir consulter les enrichissements générés automatiquement par le système pour chaque produit. Ces enrichissements incluent :

- une description détaillée conçue pour présenter les caractéristiques principales du produit
- un résumé court destiné à une présentation rapide sur les interfaces compactes
- les traductions des contenus dans les langues demandées

Ces propositions doivent être clairement présentées dans l'interface de SAP Commerce Backoffice, permettant au Responsable de l'enrichissement du produit d'accepter, modifier ou rejeter chaque suggestion.

### Détection des anomalies

Le système doit identifier automatiquement les problèmes présents dans les données des produits. Cela inclut :

- les champs obligatoires absents ou non conformes aux règles définies
- les erreurs de structure ou de format
- les incohérences sémantiques ou linguistiques

Cette détection permet au responsable de l'enrichissement du produit de corriger les défauts avant la publication et d'améliorer la qualité globale du catalogue.

## **Validation des fiches des produits**

Le responsable de l'enrichissement du produit doit pouvoir consulter un score de qualité global pour chaque fiche du produit, accompagné d'une liste des anomalies détectées. Ce score synthétise les résultats de l'analyse technique et sémantique. Sur cette base, Le responsable de l'enrichissement du produit décide d'approuver la fiche ou d'exiger des corrections avant publication.

### **2.2.3 Recueil des besoins non fonctionnels**

Les besoins non fonctionnels définissent les critères de qualité que doit respecter le système afin de garantir son efficacité, sa fiabilité et son intégration dans l'environnement existant. Ils concernent notamment les aspects liés à la performance, la sécurité, l'utilisabilité et la maintenabilité de la solution.

## **Fiabilité**

Le système doit fournir des enrichissements et des validations de qualité suffisante. Cette fiabilité repose sur des règles métier bien définies, des modèles adaptés au domaine et une validation humaine des résultats proposés.

## **Performance**

Le temps de réponse doit être acceptable pour ne pas interrompre le flux de travail du responsable de l'enrichissement du produit. Une architecture découplée, avec l'intelligence artificielle déployée dans un microservice externe, permet d'optimiser les temps de traitement sans ralentir la plateforme SAP Commerce.

## **Sécurité**

La sécurité est assurée par la protection des données échangées entre SAP Commerce et le microservice d'intelligence artificielle. Seules les informations nécessaires aux traitements d'enrichissement et de validation sont transmises afin de limiter l'exposition des données sensibles. Les échanges sont sécurisés via HTTPS et authentifiés pour garantir que seules les requêtes légitimes sont traitées par le microservice.

## **Utilisabilité**

Les résultats d'enrichissement et de validation doivent être présentés de manière claire et compréhensible dans l'interface du Backoffice. Le responsable de l'enrichissement du produit doit rapidement identifier les anomalies signalées et les enrichissements proposés, afin de pouvoir les accepter ou rejeter.

## **Maintenabilité**

L'architecture doit permettre une évolution indépendante du microservice d'intelligence artificielle sans impacter SAP Commerce. Les règles métier et les modèles doivent pouvoir être actualisés sans modifier la plateforme e-commerce existante.

## **2.3 Modélisation des besoins**

Cette section modélise le système d'enrichissement et de validation des produits au travers de trois diagrammes UML [9] complémentaires : le diagramme de cas d'utilisation expose les scénarios métier, le diagramme de séquence détaille l'orchestration temporelle des échanges et le diagramme d'activités synthétise le flux de contrôle du traitement.

### 2.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les interactions principales entre le responsable de l'enrichissement du produit et le système. Il met en évidence les fonctionnalités offertes par la solution : l'enrichissement automatique des contenus, la détection des anomalies, la consultation des résultats de validation et la prise de décision finale avant publication. Ces interactions sont illustrées dans la Figure 2.1.

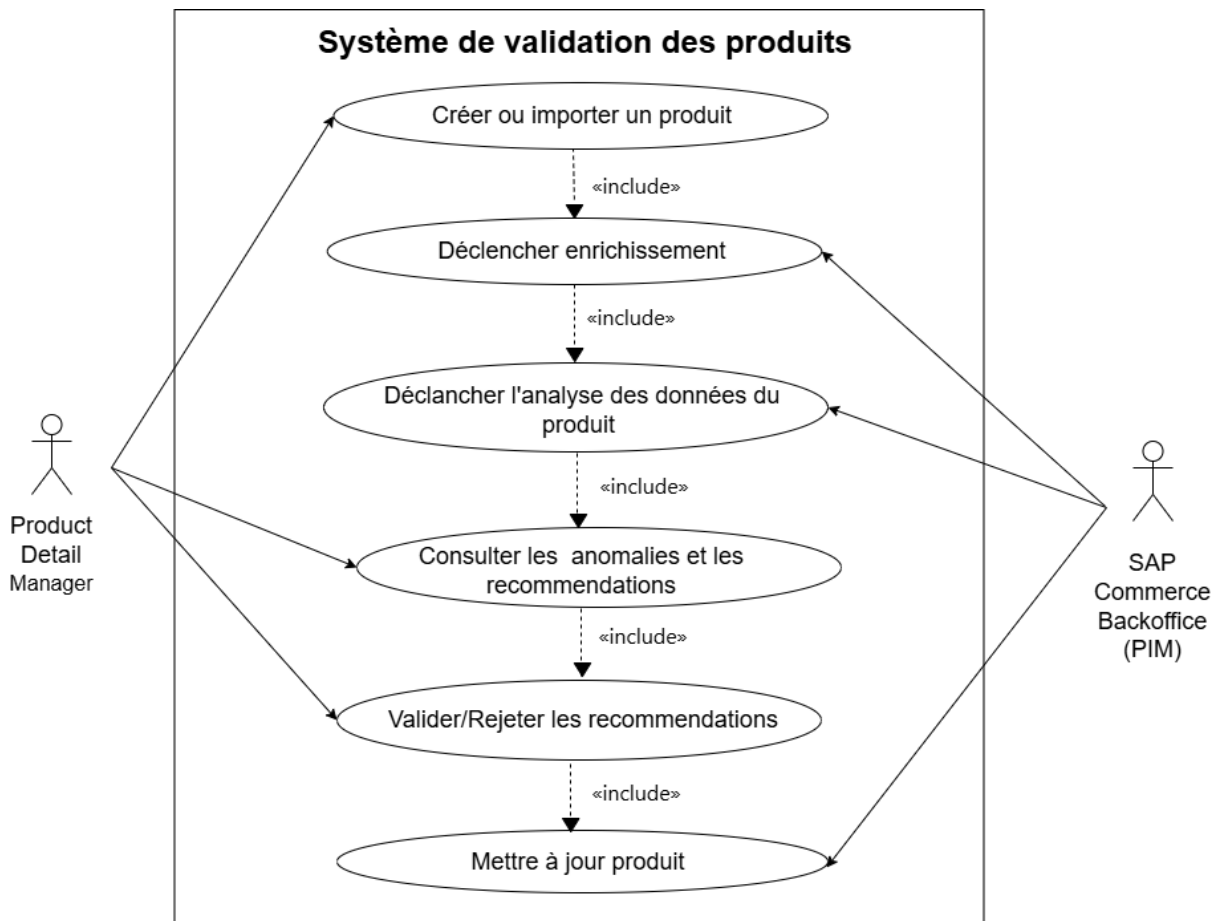


FIGURE 2.1 – Diagramme de cas d'utilisation

Ce diagramme montre que le responsable conserve un rôle central de décideur, tandis que les traitements techniques (enrichissement et validation) sont automatisés par la solution.

### 2.3.2 Diagramme de séquence

Le diagramme de séquence détaille l'ordre temporel des interactions entre les différents acteurs du système. La Figure 2.2 illustre le flux complet d'une requête depuis sa création dans SAP Commerce jusqu'à la réception de la réponse du microservice d'intelligence artificielle.

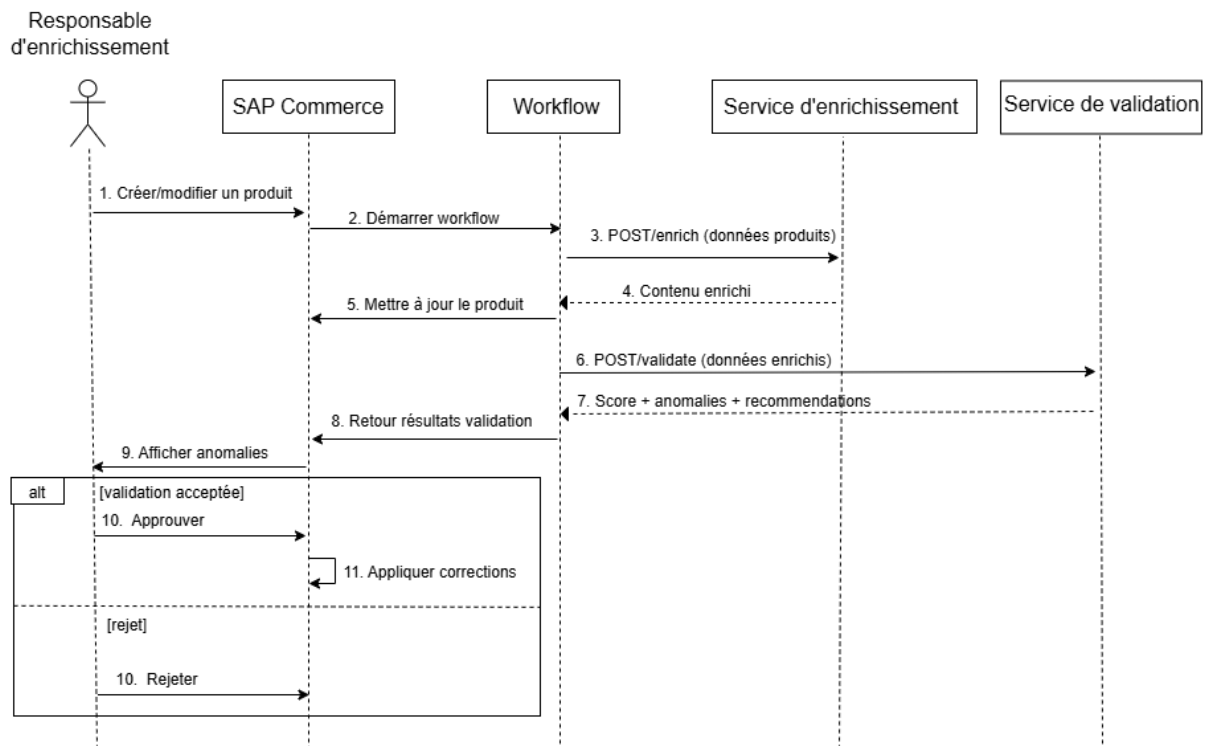


FIGURE 2.2 – Diagramme de séquence

Ce diagramme démontre la nature asynchrone et bien définie des échanges : chaque acteur (responsable, SAP Commerce, microservice) exécute ses responsabilités de manière séquentielle, facilitant ainsi le débogage et l'optimisation du système.

### 2.3.3 Diagramme d'activités

Le diagramme d'activités représente le flux global du traitement d'une fiche du produit et met en évidence les différentes étapes ainsi que les points de décision du processus. Il illustre l'enchaînement entre l'intervention du responsable de l'enrichissement du produit, l'orchestration réalisée par SAP Commerce, les traitements effectués par le microservice d'intelligence artificielle et la décision finale avant publication. Ce flux complet est présenté dans la Figure 2.3.

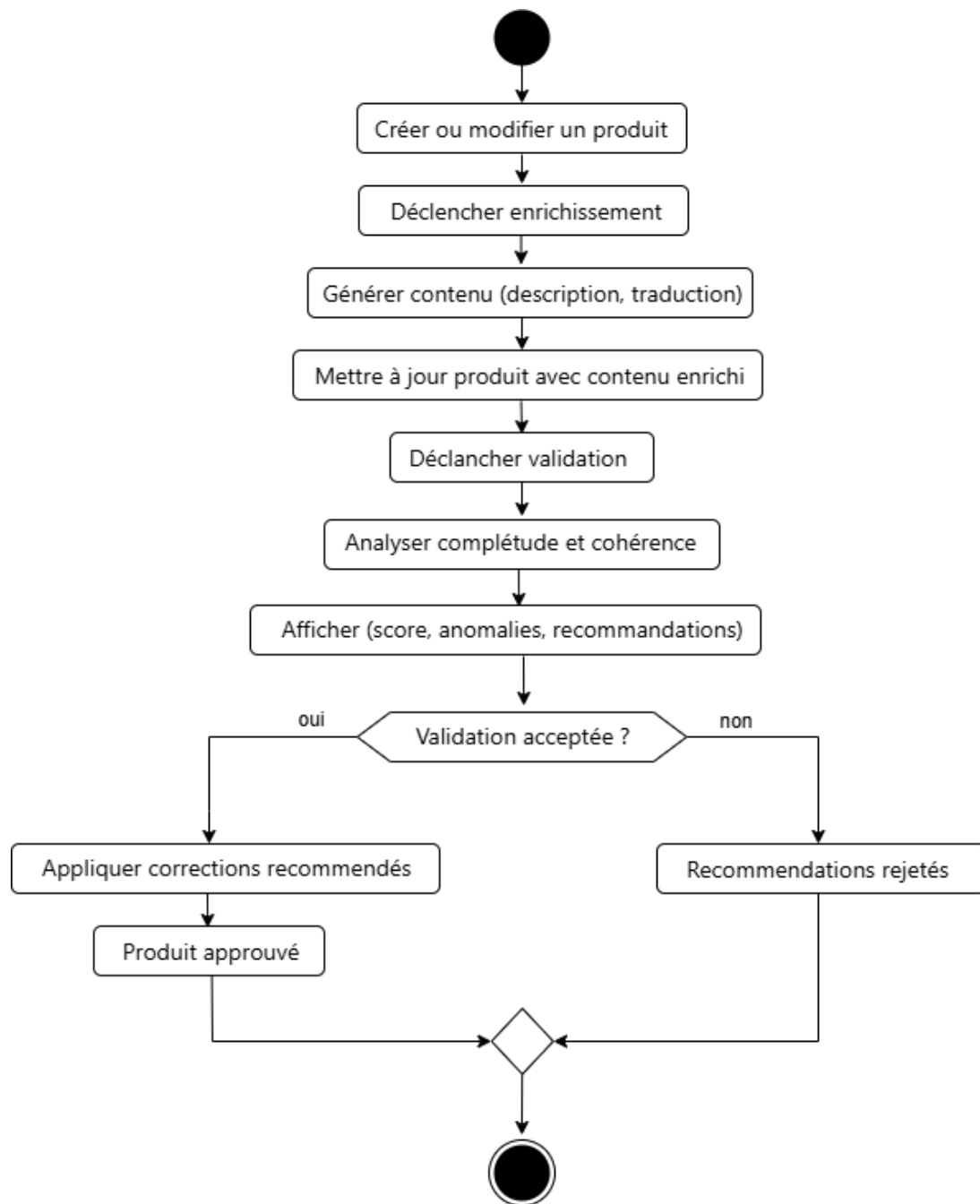


FIGURE 2.3 – Diagramme d'activités

Ce diagramme souligne l'importance de la validation humaine dans le processus : bien que le système automatise les tâches d'enrichissement et de validation technique, la décision finale de publication demeure sous le contrôle du responsable, garantissant ainsi une qualité finale acceptable par le métier.

## 2.4 Architecture globale

L'architecture du système repose sur une séparation de responsabilités entre la plateforme SAP Commerce et un microservice [10] d'intelligence artificielle indépendant. Cette approche centralise les traitements intelligents dans un composant dédié tout en intégrant les résultats dans le workflow existant de gestion des produits. L'organisation globale de cette architecture est illustrée dans la Figure 2.4.

### 2.4.1 Vue générale de l'architecture

L'architecture repose sur trois éléments principaux : SAP Commerce qui orchestre le processus métier et accueille les extensions nécessaires au déclenchement des traitements, une API REST [11] qui assure la communication entre les deux systèmes et un microservice d'intelligence artificielle externe, qui réalise les traitements d'enrichissement et de validation.

Cette approche présente plusieurs avantages. Elle concentre les traitements intelligents dans un composant indépendant, facilitant la maintenance et l'évolution de la solution. Elle permet également une séparation claire des responsabilités : SAP Commerce gère le cycle de vie des produits tandis que le microservice d'intelligence artificielle prend en charge les traitements liés à l'intelligence artificielle.

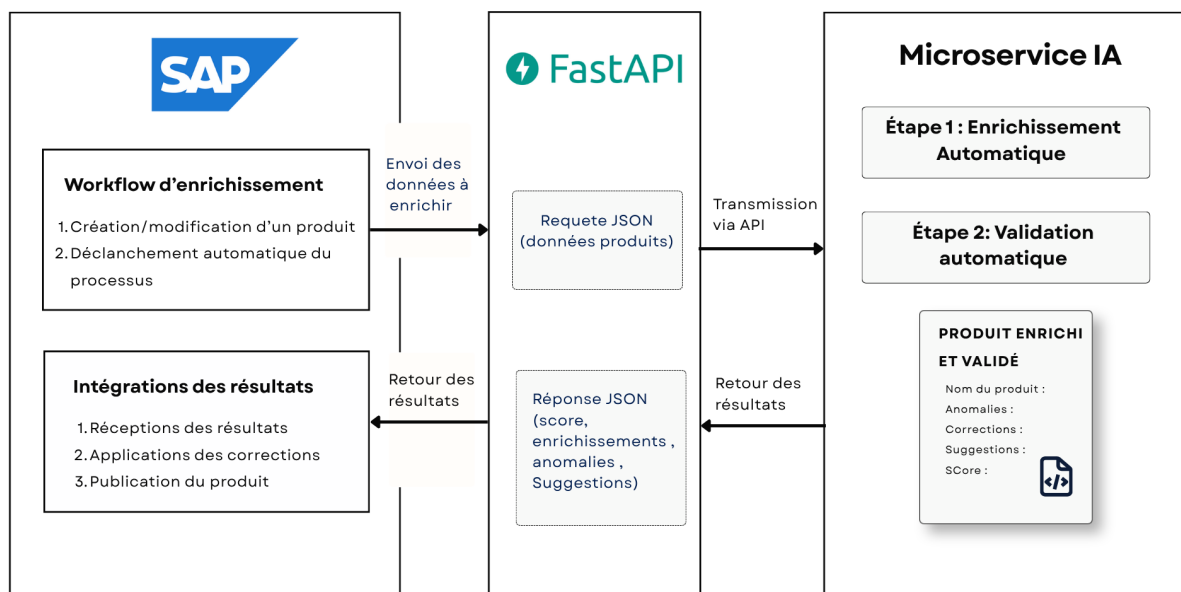


FIGURE 2.4 – Vue d'ensemble de l'architecture du système d'enrichissement intelligent

Cette architecture démontre l'efficacité d'une séparation nette entre la gestion



transactionnelle (SAP Commerce) et les traitements intelligents (microservice). Cette approche facilite la maintenance, l'amélioration et le déploiement de chaque composant indépendamment, tout en garantissant une intégration harmonieuse au sein de l'écosystème global.

## 2.4.2 Description détaillée des composants

### SAP Commerce : orchestrateur métier

SAP Commerce [6] (anciennement Hybris) constitue le cœur du système de gestion des produits. Elle assure la gestion des données produits et l'orchestration du workflow associé.

SAP Commerce accueille les événements métier relatifs à la création et la modification de produits. Des actions et services workflow ont été ajoutés pour intégrer le traitement d'enrichissement et de validation dans ce flux existant. Lorsque le responsable de l'enrichissement du produit crée ou modifie une fiche du produit, un événement déclenche l'extraction des données pertinentes (titre, description, attributs, catégorie, langue). Ces données sont nettoyées, normalisées et structurées en JSON pour transmission au microservice. À réception des résultats d'analyse, SAP Commerce les présente au responsable de l'enrichissement du produit dans l'interface du Backoffice. Le responsable de l'enrichissement du produit examine alors les enrichissements proposés, les anomalies signalées et le score de qualité, puis prend la décision d'approuver ou de corriger la fiche avant publication.

### API REST : contrat d'intégration

L'API REST assure la communication standardisée entre SAP Commerce et le microservice d'intelligence artificielle. REST a été choisi car c'est un standard pour les intégrations inter-services : simple à mettre en œuvre, adapté aux requêtes synchrones et bien supporté par les frameworks modernes.

Les données sont échangées au format JSON [12]. Les réponses structurées en JSON contiennent : les enrichissements générés, la liste des anomalies détectées et un score de qualité global.

### Microservice d'intelligence artificielle : traitements d'enrichissement et de validation

Le microservice d'intelligence artificielle, développé avec FastAPI [13] et Python [14], implémente les traitements d'enrichissement et de validation des produits. Il fonctionne selon deux phases successives : l'enrichissement d'abord, puis la validation. Cette séparation permet une progression claire du traitement et facilite le diagnostic en cas de dysfonctionnement.

### 2.4.3 Communication entre SAP Commerce et le microservice

La communication entre SAP Commerce et le microservice d'intelligence artificielle repose sur une API REST utilisant le format JSON. SAP Commerce transmet uniquement les informations nécessaires aux traitements d'enrichissement et de validation, afin de limiter l'exposition des données sensibles.

Le microservice reçoit ces données structurées, exécute les traitements demandés, puis retourne une réponse contenant les contenus enrichis, les anomalies détectées et le score de qualité associé.

### 2.4.4 Flux global de traitement

Le fonctionnement global de la solution repose sur l'échange entre SAP Commerce et le microservice d'intelligence artificielle. Le traitement d'une fiche du produit suit les étapes techniques suivantes :

1. **Préparation des données** : SAP Commerce récupère les informations nécessaires depuis la fiche du produit et prépare une structure JSON conforme au contrat défini avec le microservice d'intelligence artificielle.
2. **Transmission au microservice d'intelligence artificielle** : Les données préparées sont envoyées via une requête REST afin de déclencher les traitements d'intelligence artificielle.
3. **Traitement d'enrichissement** : Le microservice génère les contenus attendus, notamment la description détaillée, le résumé court et les traductions demandées.
4. **Traitement de validation** : Après enrichissement, le système analyse la qualité des contenus selon des règles techniques et sémantiques, puis identifie les anomalies éventuelles.
5. **Retour et intégration des résultats** : Le microservice retourne les résultats sous forme structurée. SAP Commerce présente ces informations dans le Backoffice afin de permettre au responsable de l'enrichissement du produit d'effectuer la validation finale.

Cette communication assure une séparation claire entre la gestion métier des produits dans SAP Commerce et les traitements intelligents réalisés par le microservice d'intelligence artificielle.

### 2.4.5 Justification des choix architecturaux

Cette architecture a été retenue pour plusieurs raisons.

#### Intégration progressive dans l'existant

Plutôt que de remplacer le workflow produit existant, l'architecture ajoute des extensions minimales dans SAP Commerce pour déclencher les traitements IA. Cette approche réduit le risque de perturbation du système en production.

#### Indépendance des évolutions

Le microservice peut être amélioré ou modifié sans impact sur le workflow SAP Commerce. Les règles métier et les modèles peuvent être actualisés indépendamment.

### **Traçabilité des traitements**

Les échanges entre les deux systèmes sont loggés, permettant de suivre chaque traitement et de diagnostiquer les dysfonctionnements.

### **Responsabilité humaine préservée**

Le responsable de l'enrichissement du produit conserve l'autorité finale sur la publication des fiches, garantissant un contrôle qualité manuel sur les décisions critiques.

## **2.4.6 Justification des choix technologiques**

### **SAP Commerce**

L'équipe utilise SAP Commerce pour gérer le catalogue e-commerce. Cette plateforme offre le workflow produit, les événements métier et les APIs nécessaires à l'intégration.

### **FastAPI et Python**

FastAPI est un framework léger et performant pour construire des APIs. Python offre des bibliothèques matures pour le traitement du texte et l'intégration de modèles de traitement naturel du langage, facilitant le développement rapide des traitements IA.

### **REST et JSON**

REST est le standard pour les communications inter-services. JSON offre un format léger et universel pour l'échange de données structurées.

Chaque choix correspond à une nécessité du projet et aux contraintes d'intégration avec les systèmes existants.

## **2.5 Conclusion du chapitre**

Ce chapitre a présenté la conception et l'architecture du système d'enrichissement intelligent. Nous avons formalisé les besoins fonctionnels et non fonctionnels, modélisé les interactions par des diagrammes UML et décrit l'architecture distribuée reposant sur SAP Commerce et un microservice d'intelligence artificielle externe. Cette séparation des responsabilités garantit la stabilité du système existant tout en permettant l'évolution indépendante des composants IA.

Le chapitre suivant décrit la réalisation concrète : implémentation du microservice, intégration avec SAP Commerce, résultats obtenus.

# Chapitre 3

## Module d'enrichissement et de validation des produits

### 3.1 Introduction

Après avoir présenté l'architecture globale de la solution dans le chapitre précédent, ce chapitre détaille l'implémentation technique et les algorithmes du microservice d'intelligence artificielle. Celui-ci constitue le cœur des traitements d'enrichissement et de validation des fiches produits.

Nous présentons dans un premier temps l'organisation générale du microservice ainsi que les choix technologiques retenus pour son développement. Les différentes étapes du pipeline de traitement sont ensuite décrites avant d'aborder, dans les sections suivantes, les mécanismes d'enrichissement, de traduction et de validation mis en œuvre.

### 3.2 Conception du microservice

Le microservice d'intelligence artificielle est développé comme un composant indépendant de SAP Commerce Cloud. Il centralise l'ensemble des traitements liés à l'enrichissement automatique des fiches produits, à leur traduction multilingue ainsi qu'à leur validation avant leur mise à disposition des utilisateurs.

En isolant ces traitements, l'architecture reste faiblement couplée : SAP Commerce Cloud reste responsable de la gestion des données métier et des interactions avec les utilisateurs, tandis que le microservice prend en charge l'ensemble des traitements d'intelligence artificielle. Les deux composants communiquent uniquement au travers des interfaces REST, ce qui facilite l'évolution de chacun sans remettre en cause le fonctionnement de l'autre.

### 3.2.1 Déroulement du traitement

Le microservice réalise les traitements selon une succession d’étapes, chacune ayant un objectif précis. Les données progressent d’un module à l’autre jusqu’à la production du résultat final. Ce découpage modulaire garantit l’indépendance des fonctions et simplifie les mises à jour ultérieures. Le traitement d’une requête s’effectue selon les étapes suivantes :

1. **Validation des données d’entrée** : la requête reçue est d’abord vérifiée afin de s’assurer qu’elle respecte la structure JSON attendue par le microservice. Les champs obligatoires, les types de données ainsi que le format des informations sont contrôlés avant le lancement des traitements d’intelligence artificielle.
2. **Enrichissement des contenus** : cette étape est composée de deux traitements successifs :
  - génération automatique des descriptions produits à l’aide du modèle de langage pré-entraîné, en se basant sur les caractéristiques techniques fournies dans la requête
  - traduction des contenus générés vers les langues demandées à l’aide du modèle de traduction
3. **Validation des résultats** : les contenus enrichis sont ensuite analysés afin de vérifier leur qualité. Le microservice applique les différentes règles de validation, détecte les anomalies éventuelles, propose des corrections lorsque cela est possible et calcule un score global de qualité avant de construire la réponse retournée à SAP Commerce Cloud.

### 3.2.2 Choix technologiques

Le microservice est développé en Python [14], choix justifié par l’écosystème mature du traitement du langage naturel et de l’intelligence artificielle. PyTorch [?], Transformers [?] et les outils d’adaptation fine-tuning sont des références incontournables dans ce domaine.

Pour l’interface REST, FastAPI [13] a été retenu. Ce framework valide automatiquement les données entrantes, génère une documentation interactive et sérialise les réponses en JSON avec un minimum de code. Pydantic [?] assure la validation stricte des requêtes et des réponses.

| Bibliothèque          | Rôle dans le projet                      |
|-----------------------|------------------------------------------|
| FastAPI               | Exposition de l’interface REST [13]      |
| Pydantic              | Validation des requêtes et réponses JSON |
| Transformers          | Chargement des modèles pré-entraînés [?] |
| PEFT                  | Fine-tuning efficace avec LoRA [?]       |
| Ollama                | Exécution locale du modèle Qwen [?]      |
| Langdetect            | Détection automatique de la langue [?]   |
| Sentence Transformers | Analyse de similarité sémantique [?]     |



### 3.3 Module d’enrichissement intelligent

L’objectif de l’enrichissement est d’améliorer la qualité des fiches des produits en générant automatiquement des descriptions détaillées et cohérentes à partir des caractéristiques techniques fournies. Cette automatisation réduit le temps de saisie manuel et assure une présentation uniforme des produits sur les différents canaux de vente.

#### 3.3.1 Génération automatique des descriptions

##### Principes des grands modèles de langage

Les grands modèles de langage (LLM pour *Large Language Models*) sont des réseaux de neurones profonds entraînés sur d’énormes quantités de texte. Ils apprennent à prédire le mot suivant dans une phrase, facilitant ainsi la génération de texte cohérent et contextuel.

Pour le projet, un modèle de langage est utilisé pour générer des descriptions de produits détaillées à partir de caractéristiques techniques simples.

##### Comparaison et sélection du modèle

Plusieurs modèles de langage existaient au moment du projet : GPT-4, Claude, Llama, Mistral et Qwen2.5 [?]. Le tableau ci-dessous les compare selon les critères importants pour une utilisation industrielle.

| Modèle     | Exécution   | Coût        | Confidentialité        | Intégration |
|------------|-------------|-------------|------------------------|-------------|
| GPT-4      | API externe | Élevé       | Données chez OpenAI    | Facile      |
| Claude     | API externe | Élevé       | Données chez Anthropic | Facile      |
| Llama      | Local/Cloud | Moyen       | Complète               | Moyenne     |
| Mistral    | Local/Cloud | Faible      | Complète               | Bonne       |
| Qwen2.5 7B | Local       | Très faible | Complète               | Excellente  |

Qwen2.5 7B a été choisi comme meilleur compromis. Ce modèle de 7 milliards de paramètres peut s’exécuter localement sur du matériel grand public, sans envoyer les données produits vers des services externes. Il est performant pour la génération de texte technique et son intégration dans le microservice est directe grâce à Ollama [?].

##### Exécution locale avec Ollama

L’exécution du modèle de génération est assurée par Ollama [?], une plateforme permettant de déployer et d’utiliser des modèles de langage de grande taille en local. Ollama fournit une interface REST qui facilite l’intégration des modèles au sein du microservice tout en simplifiant leur gestion et leur mise à jour.

Le recours à une exécution locale répond à plusieurs exigences du projet. Le choix d’un modèle local répond aux impératifs de confidentialité propres au secteur industriel en évitant l’envoi des informations produits vers des services d’intelligence artificielle hébergés sur le cloud. Les données demeurent ainsi entièrement dans l’environnement de l’entreprise.

Par ailleurs, cette approche supprime la dépendance vis-à-vis de services externes et de leur disponibilité. Le microservice peut continuer à fonctionner sans connexion à une API distante, ce qui améliore sa robustesse et assure une meilleure maîtrise des temps de réponse.

### Génération du résumé et de la description

Quand une requête arrive, le microservice commence par vérifier si les champs **récapitulatif** et **description** existent déjà. Si c’est le cas, ils sont conservés afin de préserver les contenus rédigés manuellement. Seuls les champs absents font l’objet d’une génération automatique.

À partir des informations disponibles sur le produit (nom, catégorie, marque et caractéristiques techniques), le microservice construit un prompt adapté au contexte. Celui-ci fournit au modèle l’ensemble des informations nécessaires ainsi que les contraintes de rédaction à respecter. Le résumé doit être constitué d’une seule phrase de quinze mots maximum, tandis que la description doit comporter entre deux et quatre phrases rédigées dans un style professionnel adapté au secteur industriel. Le système harmonise ainsi la qualité rédactionnelle sur l’ensemble du catalogue tout en limitant les variations entre les différentes fiches produits.

Voici un exemple du type de prompt transmis au modèle :

#### Exemple de prompt envoyé au modèle

Génère une description professionnelle pour ce produit :

Nom : Moteur électrique

Catégorie : Motorisation industrielle

Caractéristiques :

— Puissance : 50 W

— Tension : 220 V

— Protection : IP55

La description doit être claire, complète et adaptée à un public industriel.

Le prompt est ensuite transmis au modèle Qwen2.5, exécuté localement via Ollama. La

réponse est retournée sous la forme d’un objet JSON contenant les deux champs générés. Le microservice extrait ces informations avant de les intégrer dans la réponse destinée à SAP Commerce Cloud.

Avant de renvoyer le résultat, plusieurs contrôles sont réalisés pour assurer la pertinence des descriptions générées. Le microservice vérifie notamment la présence des champs attendus, la validité du format JSON et le respect de la longueur maximale imposée au résumé.

En cas d’erreur de communication ou de génération, une nouvelle tentative est automatiquement effectuée. Si aucune réponse exploitable n’est obtenue après plusieurs essais, le titre du produit est utilisé comme valeur par défaut afin de garantir qu’aucune fiche produit ne soit retournée avec des champs textuels vides.

Cette gestion des erreurs fiabilise l’enrichissement et prévient toute interruption du flux de données. Elle permet également de limiter les risques liés à des réponses incomplètes ou mal formatées sans interrompre le traitement des produits.

### 3.3.2 Module de traduction multilingue

SAP Commerce Cloud doit alimenter des sites e-commerce dans plusieurs pays. Chaque fiche produit doit être disponible en français, allemand, italien, espagnol, polonais, tchèque, hongrois, indonésien, russe, coréen, japonais, chinois simplifié, hindi, portugais, chinois traditionnel et certaines variantes régionales.

La traduction manuelle serait extrêmement coûteuse et lente. Une traduction automatique de qualité est donc essentielle.

#### Comparaison des modèles de traduction

Plusieurs modèles de traduction neuronale existent : MarianMT, mBART-50, M2M100 et NLLB [?]. Le modèle NLLB (No Language Left Behind) de Meta a été sélectionné pour sa couverture de 200 langues, ses performances élevées et sa compatibilité avec le fine-tuning.

La version utilisée est `facebook/nllb-200-distilled-600M`, une version compacte de 616 millions de paramètres qui offre un bon équilibre entre qualité et ressources requises.

#### Présentation générale de NLLB

NLLB [?] est un modèle Transformer de type Seq2Seq (séquence à séquence). Il prend une phrase en entrée et génère sa traduction en sortie. Le modèle a été entraîné sur des données de traduction couvrant 200 langues.

Un point clé de NLLB est l’utilisation des codes FLORES-200. Chaque langue possède un code spécifique (par exemple `fra_Latn` pour le français, `deu_Latn` pour l’allemand). Le microservice convertit les codes de langues SAP Commerce Cloud vers ces codes FLORES avant la traduction.

#### Fine-tuning avec LoRA

Le modèle NLLB pré-entraîné fonctionne correctement, mais un fine-tuning a été réalisé pour adapter le modèle aux spécificités du projet : contexte industriel, vocabulaire technique, style de rédaction des fiches produits.

Le fine-tuning complet d’un modèle de 616 millions de paramètres nécessiterait énormément de mémoire GPU. Pour contourner ce problème, la technique LoRA (Low-Rank Adaptation) [?] a été utilisée. LoRA gèle la majorité des paramètres du modèle et n’entraîne que des petites matrices d’adaptation. Dans ce projet, seuls 1,18 million de paramètres ont été rendus entraînaibles, soit environ 0,19 % du total. Cette approche réduit dramatiquement les besoins en ressources tout en conservant de bonnes performances.

## Jeu de données d’entraînement

Les données proviennent directement du catalogue SAP Commerce Cloud d’un client B2B. Des phrases en anglais ont été extraites manuellement avec leurs traductions vers 16 langues cibles.

La structure du fichier JSON est simple : chaque entrée contient une phrase source en anglais et ses traductions. Pour enrichir l’apprentissage, des paires bidirectionnelles ont été générées : chaque traduction vers une langue cible est aussi utilisée comme source pour traduire vers l’anglais.

Après génération bidirectionnelle, le jeu de données contenait environ 21 800 paires brutes. Il a été réduit à 5 000 exemples pour l’entraînement et 500 pour la validation, chacun limité à 256 tokens maximum. Cette réduction a été nécessaire en raison des contraintes de ressources GPU.

Les langues incluses dans le fine-tuning sont : français, allemand, italien, espagnol, polonais, tchèque, hongrois, indonésien, russe, coréen, japonais, chinois simplifié, hindi, portugais et chinois traditionnel.

## Configuration de l’entraînement

Le fine-tuning du modèle NLLB a été réalisé à l’aide de la méthode LoRA. Les hyperparamètres ont été choisis de manière à obtenir un compromis entre qualité d’apprentissage et contraintes matérielles. Le Tableau 3.1 présente les principaux paramètres retenus ainsi que leur justification.

| Paramètre         | Valeur    | Définition et justification                                                                                                                                                  |
|-------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rang LoRA ( $r$ ) | 8         | Définit la taille des matrices ajoutées par LoRA. Une valeur de 8 offre un bon compromis entre capacité d’adaptation et nombre de paramètres entraînés.                      |
| LoRA Alpha        | 16        | Contrôle l’influence des couches LoRA sur le modèle. Cette valeur permet une adaptation efficace tout en préservant les connaissances du modèle pré-entraîné.                |
| Learning rate     | $10^{-4}$ | Détermine l’amplitude des mises à jour des poids. Cette valeur favorise une convergence progressive de l’entraînement.                                                       |
| Nombre d’époques  | 3         | Correspond au nombre de passages sur l’ensemble des données d’entraînement. Trois époques ont été retenues pour adapter le modèle sans prolonger inutilement l’entraînement. |
| Batch effectif    | 2         | Représente le nombre d’exemples utilisés à chaque mise à jour des poids. Cette valeur limite la consommation mémoire tout en assurant un apprentissage stable.               |

TABLE 3.1 – Principaux hyperparamètres utilisés lors du fine-tuning du modèle NLLB

L’entraînement a été réalisé sur un GPU de type GTX disposant de 4 Go de mémoire

vidéo. L’utilisation de LoRA, associée à une taille de lot réduite et à l’accumulation des gradients, a permis d’adapter le modèle NLLB dans ces contraintes matérielles sans nécessiter d’infrastructure de calcul spécialisée.

### Résultats du fine-tuning

Le score BLEU (Bilingual Evaluation Understudy) [?] est une métrique de référence qui mesure la similarité n-grammatique entre la traduction générée par le modèle et une traduction de référence produite par un humain.

L’évaluation sur 7 langues (30 échantillons chacune) donne un score BLEU moyen de 28.4, en progression significative par rapport aux 23,5 du modèle de base. Bien que modeste face aux 35 BLEU des modèles les plus performants, ce score confirme que le fine-tuning LoRA [?] sur un jeu de 5 000 paires adapte efficacement le modèle au vocabulaire technique et au style des fiches produits, validant ainsi son déploiement en production.

### Fonctionnement du module de traduction dans le microservice

Le module de traduction intégré au microservice assure la génération des différentes versions linguistiques des contenus produits tout en garantissant la cohérence et la fiabilité des données échangées avec SAP Commerce Cloud. Afin d’optimiser les temps de réponse, les ressources nécessaires au traitement linguistique sont initialisées au démarrage du microservice et maintenues en mémoire durant son exécution. Cette approche permet d’éviter les opérations répétitives de chargement et d’améliorer les performances globales du système.

Le processus de traduction repose sur plusieurs étapes principales :

- **Adaptation des paramètres linguistiques** : les codes de langues utilisés par SAP Commerce Cloud sont convertis vers le format attendu par le module de traduction. Cette étape assure l’interopérabilité entre les deux environnements et garantit une interprétation correcte des langues cibles.
- **Traitement des contenus multilingues** : les différents champs textuels du produit, notamment le récapitulatif et la description, sont traités séparément pour chaque langue demandée. Cette organisation permet de préserver la structure des informations et d’assurer une correspondance précise entre les contenus sources et leurs variantes linguistiques.
- **Gestion des erreurs de traitement** : un mécanisme de secours est mis en place afin d’assurer la continuité du processus en cas d’indisponibilité ou d’échec lors de la

génération d’une traduction. Dans ce cas, la version source du contenu est conservée afin d’éviter l’introduction de données incomplètes dans le catalogue produit.

- **Centralisation des résultats** : les différentes versions linguistiques obtenues sont regroupées dans une structure de données unique avant leur transmission vers SAP Commerce Cloud. Cette organisation facilite la mise à jour cohérente des informations du produit au sein du catalogue.

Ainsi, le module de traduction contribue à automatiser la gestion des contenus multilingues tout en maintenant la cohérence des informations échangées avec SAP Commerce Cloud. Son intégration au sein du microservice permet de centraliser le traitement linguistique, de simplifier les échanges avec le catalogue produit et d’assurer une mise à disposition fiable des différentes versions linguistiques.

### 3.3.3 Structure de la réponse du service d’enrichissement

À l’issue du processus d’enrichissement, le microservice retourne une réponse structurée contenant les informations générées ainsi que les métadonnées nécessaires à leur intégration dans SAP Commerce Cloud. Cette organisation permet de séparer les contenus enrichis des informations de contrôle et facilite le traitement automatique de la réponse par le système appelant.

La réponse de l'API d'enrichissement est composée principalement des éléments suivants :

- **Identifiant du produit** : permet d'assurer la correspondance entre les données enrichies retournées et le produit concerné dans SAP Commerce Cloud.
- **Contenus générés** : regroupe les champs textuels enrichis, notamment le récapitulatif et la description du produit dans la langue source.
- **Traductions associées** : contient les différentes versions linguistiques générées pour les langues cibles demandées.
- **Informations de traitement** : fournit l'état d'exécution de la demande ainsi que les éventuels messages liés au déroulement du processus.

La Figure 3.3.3 présente un exemple simplifié de la structure JSON retournée par l'API d'enrichissement.

#### Exemple de réponse JSON de l'API d'enrichissement

```
{
  "productCode": "PROD-001",
  "status": "SUCCESS",
  "enrichment": {
    "summary": "Résumé généré du produit",
    "description": "Description détaillée générée du produit"
  },
  "translations": {
    "fr": {
      "summary": "Résumé traduit",
      "description": "Description traduite"
    },
    "de": {
      "summary": "Übersetzte Zusammenfassung",
      "description": "Übersetzte Beschreibung"
    }
  }
}
```

Cette structure permet au système SAP Commerce Cloud de récupérer les résultats d'enrichissement de manière claire et standardisée. La séparation entre les contenus



générés, les traductions et les informations de traitement facilite également l’évolution du service et son intégration avec d’autres applications exploitant le catalogue produit.

### 3.3.4 Module de validation : contrôle de qualité automatique

Le module de validation assure que chaque fiche produit respecte un certain nombre de critères avant d’être utilisée dans SAP Commerce Cloud. Il détecte les erreurs, les données manquantes et les incohérences.

Le pipeline de validation fonctionne en quatre couches successives, chacune contrôlant des aspects différents.

#### Couche 1 : Validation structurelle

La première couche du processus de validation a pour objectif de vérifier la conformité structurelle des données reçues avant toute analyse de contenu. Elle constitue une étape de filtrage indispensable permettant de détecter les erreurs susceptibles de compromettre les traitements réalisés par les couches suivantes. Une fiche produit qui ne satisfait pas ces contrôles est considérée comme incomplète ou invalide et ne peut être analysée de manière fiable.

Les principaux contrôles effectués sont les suivants :

- **Vérification des champs obligatoires** : contrôle de la présence des informations indispensables à l’identification et à l’exploitation du produit, telles que son identifiant, son nom ou les attributs requis.
- **Contrôle des types de données** : validation de la cohérence des types attendus pour chaque attribut. Par exemple, un prix doit être représenté par une valeur numérique tandis qu’une URL doit être stockée sous forme de chaîne de caractères.
- **Validation des formats et des contraintes** : vérification de règles simples telles que la présence d’un prix positif, la longueur minimale des descriptions ou encore le respect du format des adresses URL.
- **Vérification des langues déclarées** : contrôle de la validité des codes de langue utilisés afin d’assurer leur compatibilité avec les traitements multilingues du microservice.

Cette première couche garantit ainsi que seules des données correctement structurées sont transmises aux traitements de validation avancés.

#### Couche 2 : Validation sémantique

Une fois la conformité structurelle vérifiée, le microservice procède à une analyse du contenu des informations textuelles. Contrairement à la validation structurelle, qui s’intéresse uniquement au format des données, cette seconde couche évalue leur qualité linguistique et leur cohérence sémantique.

Cette analyse repose sur plusieurs mécanismes complémentaires.

- **Détection automatique de la langue** : la bibliothèque `langdetect` [?] identifie la langue réellement utilisée dans chaque contenu textuel afin de vérifier qu’elle correspond à la langue attendue pour le champ concerné. Une incohérence peut révéler une erreur de traduction, une mauvaise affectation des contenus multilingues ou une inversion des langues.
- **Analyse grammaticale et orthographique** : les textes sont analysés à l’aide de la bibliothèque `LanguageTool` [?], qui détecte les erreurs d’orthographe, de grammaire, de ponctuation et certaines maladresses de formulation susceptibles de diminuer la qualité éditoriale de la fiche produit.
- **Analyse contextuelle** : certaines anomalies ne peuvent être détectées uniquement à partir de règles grammaticales. Le modèle de langage Qwen2.5 [?] est donc sollicité afin d’évaluer le contenu dans son contexte et d’identifier des formulations peu naturelles, des incohérences rédactionnelles ou des expressions pouvant être améliorées tout en conservant le sens initial du texte.

Ces différents contrôles sont appliqués aussi bien aux contenus d’origine qu’aux descriptions traduites automatiquement par le modèle NLLB. Le module maintient ainsi une rigueur linguistique constante, quelle que soit la langue cible.

### Couche 3 : Corrections automatiques et suggestions

La troisième couche du processus de validation est dédiée à la correction des anomalies détectées lors des étapes précédentes. Afin de garantir un équilibre entre automatisation et contrôle humain, le microservice distingue les corrections pouvant être appliquées sans risque de celles nécessitant une validation par le responsable de l’enrichissement des produits. Deux catégories de traitements sont ainsi mises en œuvre :

- **Corrections automatiques** : elles concernent les anomalies de faible risque pouvant être résolues sans intervention humaine. Ces corrections améliorent la qualité technique des données sans modifier leur signification. Elles comprennent notamment le nettoyage des balises HTML incorrectes à l’aide de `BeautifulSoup` [?], la suppression des espaces superflus, la suppression des URLs locales ou invalides, la conversion des liens `http` vers `https` ainsi que la normalisation du format des unités techniques et de certains caractères. Une fois appliquées, ces modifications sont enregistrées directement dans SAP Commerce Cloud afin d’améliorer la qualité des fiches produits.
- **Suggestions de correction** : les anomalies nécessitant une compréhension du contexte ne sont pas corrigées automatiquement. Elles donnent lieu à des

recommandations destinées au responsable de l’enrichissement des produits. Ces suggestions sont générées à partir des analyses réalisées par **LanguageTool** et par le modèle de langage Qwen2.5, qui propose une reformulation plus naturelle tout en préservant le sens du texte. Chaque recommandation transmise à SAP Commerce Cloud précise le champ concerné, l’anomalie détectée, la correction proposée ainsi que son niveau de sévérité. L’utilisateur conserve ainsi la décision finale et peut accepter, modifier ou refuser chaque proposition avant son application.

Le responsable de l’enrichissement conserve ainsi le contrôle sur les modifications proposées et peut accepter, modifier ou refuser chaque suggestion avant son application. Cette séparation entre corrections automatiques et recommandations permet de concilier efficacité, fiabilité et maîtrise des contenus éditoriaux.

## Couche 4 : Calcul du score de qualité

La dernière couche du processus de validation consiste à synthétiser l’ensemble des analyses réalisées précédemment sous la forme d’un score de qualité global. Ce score fournit une mesure quantitative permettant d’évaluer rapidement le niveau de conformité d’une fiche produit et de faciliter sa prise en charge par les utilisateurs métier.

Le calcul débute avec un score maximal de 100 points. Chaque anomalie détectée lors des différentes étapes de validation entraîne ensuite l’application d’une pénalité dont la valeur dépend de son niveau de sévérité. Plus une anomalie est susceptible d’affecter la qualité ou l’exploitabilité de la fiche produit, plus la pénalité associée est importante.

Cette approche présente plusieurs avantages. D’une part, elle distingue les erreurs critiques, qui empêchent l’utilisation correcte des données, des anomalies mineures ayant principalement un impact sur la qualité éditoriale. D’autre part, elle fournit un indicateur unique facilitant le suivi de l’évolution de la qualité des produits au fil des enrichissements successifs.

Le score obtenu est également utilisé pour déterminer le statut global de validation de la fiche produit. Selon les anomalies détectées et le score final obtenu, une fiche peut être considérée comme conforme, conforme avec réserves ou non conforme. Cette classification permet au responsable de l’enrichissement d’identifier rapidement les produits nécessitant une intervention prioritaire.

Le tableau 3.2 présente les pénalités appliquées en fonction du niveau de sévérité des anomalies détectées.

TABLE 3.2 – Pondération des anomalies selon leur niveau de sévérité

| Sévérité | Pénalité appliquée | Exemple d’anomalie                            |
|----------|--------------------|-----------------------------------------------|
| CRITICAL | -15 points         | Donnée obligatoire manquante, prix invalide   |
| HIGH     | -10 points         | Langue incorrecte, contenu HTML mal formé     |
| MEDIUM   | -5 points          | Erreur grammaticale ou orthographique         |
| LOW      | -1 point           | Titre trop long, caractères spéciaux inutiles |

La notation convertit les diagnostics techniques en un indicateur métier immédiatement exploitable par SAP Commerce Cloud. Il facilite la priorisation des corrections, le suivi de la qualité des catalogues produits et l’automatisation de certaines décisions métier tout en conservant une interprétation simple pour les utilisateurs.

### 3.3.5 Standardisation de la réponse de validation

Une fois le cycle de contrôle achevé, le microservice compile les résultats dans un format JSON normalisé afin d’assurer une intégration fluide avec SAP Commerce Cloud.

Cette réponse constitue le socle décisionnel permettant au Product Manager d'évaluer la maturité d'une fiche produit avant sa publication.

La Figure 3.3.5 présente un exemple de la structure JSON retournée par le module de validation.

#### Exemple de réponse JSON du module de validation

```
{
  "productCode": "PROD-001",
  "score": 85,
  "anomalies": [
    {
      "field": "description",
      "severity": "MEDIUM",
      "message": "Erreur d'orthographe détectée",
      "suggestion": "moteur électrique"
    }
  ],
  "autoCorrections": [
    {
      "field": "content",
      "action": "HTML_CLEANUP",
      "applied": true
    }
  ]
}
```

L'analyse de cette structure montre que le microservice ne se limite pas à retourner un score global de qualité. La réponse fournit également des informations détaillées sur les anomalies détectées, notamment le champ concerné, leur niveau de sévérité ainsi que, lorsque cela est possible, une proposition de correction ou la trace des corrections appliquées automatiquement. Cette structuration facilite l'exploitation des résultats par SAP Commerce Cloud, qui peut présenter des recommandations précises au responsable de l'enrichissement des produits et assurer la traçabilité des traitements réalisés par le module de validation.

## 3.4 Conclusion du chapitre

Ce chapitre a présenté la conception et le fonctionnement du microservice d’intelligence artificielle développé dans le cadre de ce projet. Après avoir décrit son architecture générale et les technologies retenues, les différents modules qui le composent ont été détaillés, depuis la génération automatique des descriptions jusqu’à la traduction multilingue et au contrôle de la qualité des fiches produits.

Les mécanismes mis en œuvre reposent sur l’utilisation conjointe de modèles d’intelligence artificielle spécialisés et de règles de validation permettant d’automatiser une grande partie des opérations d’enrichissement tout en garantissant la cohérence et la fiabilité des données produites. L’organisation du module de validation en plusieurs couches successives assure une séparation claire des responsabilités, depuis la vérification structurelle des données jusqu’au calcul d’un score global de qualité, en passant par l’analyse sémantique et la gestion des corrections.

L’approche retenue permet ainsi de disposer d’un microservice modulaire, indépendant de SAP Commerce Cloud et facilement évolutif, capable de centraliser les traitements d’intelligence artificielle tout en exposant des interfaces REST standardisées pour communiquer avec le système d’information.

Le chapitre suivant est consacré à l’intégration de ce microservice au sein de SAP Commerce Cloud. Il présente les différents développements réalisés sur la plateforme, les mécanismes d’échange entre les deux composants ainsi que leur intégration dans les processus métier de gestion et d’enrichissement des fiches produits.

# Chapitre 4

## Intégration dans SAP Commerce Cloud

### 4.1 Introduction

L'utilité des fonctions développées dépend de leur intégration effective dans la plateforme SAP Commerce Cloud [6], où elles peuvent être exploitées directement par les utilisateurs métier et s'insérer dans les processus de gestion des produits.

Ce chapitre détaille les étapes de cette intégration. Il présente tout d'abord l'architecture de SAP Commerce Cloud afin de comprendre l'organisation de la plateforme et les mécanismes sur lesquels repose son extensibilité. Il décrit ensuite la communication entre SAP Commerce Cloud et le microservice via des services REST, les extensions apportées au modèle de données pour prendre en charge les nouveaux objets métier ainsi que les interfaces développées dans Backoffice pour permettre aux utilisateurs de piloter les opérations d'enrichissement et de validation. Enfin, il détaille l'automatisation des traitements à travers les workflows ainsi que le système de notifications et de traçabilité mis en place pour assurer le suivi des opérations.

Nous présentons ici l'automatisation des flux d'enrichissement et de validation dans le respect de l'architecture standard de la plateforme.

### 4.2 Architecture logicielle de SAP Commerce Cloud

Avant de présenter les mécanismes d'intégration du microservice d'intelligence artificielle, il est nécessaire de comprendre l'architecture logicielle de SAP Commerce Cloud [6]. L'architecture modulaire de la plateforme assure une séparation nette des responsabilités entre les différents composants. Cette organisation facilite le



développement et la maintenance du système tout en offrant les points d'extension nécessaires à l'intégration de nouvelles fonctionnalités. Cette analyse structurelle permet de justifier les choix d'implémentation adoptés.

### 4.2.1 Fondamentaux de SAP Commerce Cloud

SAP Commerce Cloud [6] est une plateforme complète de gestion du commerce électronique et du commerce unifié, conçue pour orchestrer l'intégralité du cycle de vie des produits et des commandes. Au-delà de son rôle de vitrine marchande, SAP Commerce centralise la gestion du catalogue, des prix, des stocks, des promotions et l'exécution des commandes. C'est un système d'information stratégique pour le secteur du commerce de détail et de la distribution en ligne.

Comme tout système d'information complexe, SAP Commerce s'appuie sur une organisation structurée du code, que l'on désigne par le terme d'architecture. L'architecture d'une application logicielle décrit comment le système est décomposé en composants, comment ces composants interagissent et quelles responsabilités incombent à chacun. Une bonne architecture facilite la compréhension du code, la maintenance évolutive, les tests automatisés et la répartition du travail au sein d'équipes de développement.

SAP Commerce adopte une architecture en couches, un pattern architectural reconnu et très largement utilisé dans l'industrie logicielle. Le principe fondamental de cette approche est la séparation des préoccupations : chaque couche gère un aspect distinct du système, et les dépendances entre couches sont strictement contrôlées. Cette organisation rend possible la modification d'une couche sans impact sur les autres. Par exemple, remplacer la base de données relationnelle par une base NoSQL n'affecterait pas la couche métier, car elle communique avec la persistance via une abstraction (la couche d'accès aux données). De même, changer l'interface utilisateur n'impacte pas la logique métier si cette dernière est bien encapsulée.

L'architecture en couches apporte une structure rigoureuse au projet. Premièrement, elle permet à plusieurs développeurs de travailler en parallèle sur différentes couches sans créer de conflits. Deuxièmement, elle facilite les tests : chaque couche peut être testée indépendamment des autres. Troisièmement, elle rend le système extensible : ajouter de nouvelles fonctionnalités devient une question de créer de nouveaux composants dans les couches appropriées, plutôt que de modifier du code existant. Enfin, elle favorise la réutilisabilité : une logique métier bien encapsulée dans une couche Service peut être appelée par plusieurs interfaces différentes (Backoffice, API REST, batch, etc.).

### 4.2.2 Les couches de SAP Commerce

L'architecture en couches de SAP Commerce Cloud est illustrée par la figure 4.1. Chaque couche possède une responsabilité bien définie et communique uniquement avec les couches adjacentes, ce qui favorise la modularité, la maintenabilité et l'évolutivité de la plateforme.

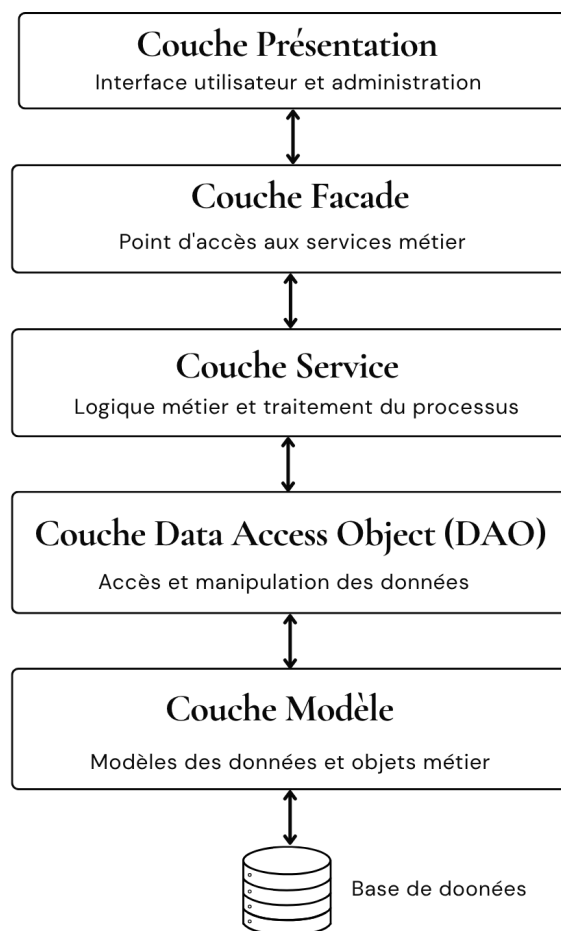


FIGURE 4.1 – Architecture en couches de SAP Commerce Cloud

Comme le montre la figure 4.1, les requêtes émises par les utilisateurs transitent

successivement par les différentes couches avant d'accéder aux données persistées. Chaque couche assure un rôle spécifique dans le traitement des opérations métier.

### **Couche Présentation**

La couche Présentation constitue le point d'interaction entre les utilisateurs et la plateforme. Dans SAP Commerce Cloud, cette couche est principalement représentée par le Backoffice, qui permet aux utilisateurs métier d'administrer les catalogues, de gérer les produits et d'exécuter les différentes opérations disponibles. Son rôle est de présenter les informations, de recueillir les actions de l'utilisateur et de transmettre les demandes aux couches inférieures.

### **Couche Facade**

La couche Facade agit comme un intermédiaire entre la présentation et la logique métier. Elle simplifie les échanges avec les services en proposant une interface unifiée aux composants de la couche Présentation. Cette couche permet également de centraliser les appels aux différents services nécessaires à l'exécution d'une opération.

### **Couche Service**

La couche Service regroupe la logique métier de l'application. Elle implémente les traitements nécessaires au fonctionnement de la plateforme et coordonne les différentes opérations réalisées lors d'un processus métier. Les services sont indépendants de l'interface utilisateur, ce qui permet leur réutilisation dans différents contextes d'exécution.

### **Couche Data Access Object (DAO)**

La couche DAO assure l'accès aux données persistées. Elle constitue une abstraction entre la logique métier et le mécanisme de persistance en encapsulant les opérations de lecture, de création, de modification et de suppression des données. Cette séparation permet d'isoler la logique métier des détails liés au stockage.

### **Couche Modèle**

La couche Modèle représente les entités métier manipulées par SAP Commerce Cloud. Ces entités, appelées *ItemTypes*, définissent la structure des données persistées ainsi que les relations existantes entre elles. Elles constituent la base sur laquelle reposent les différentes opérations réalisées par les couches supérieures.

L'organisation en couches de SAP Commerce Cloud facilite ainsi le développement de nouvelles fonctionnalités tout en préservant une séparation claire des responsabilités. Cette architecture constitue le socle sur lequel s'appuie l'intégration du microservice d'intelligence artificielle présentée dans les sections suivantes.

## 4.3 Communication entre SAP Commerce Cloud et le module d'intelligence artificielle

L'intégration du module d'intelligence artificielle dans SAP Commerce Cloud [6] repose sur une communication basée sur des API REST [11]. SAP Commerce exploite ainsi les fonctions d'IA externes sans compromettre la séparation des responsabilités entre les composants. La plateforme SAP Commerce reste responsable de la gestion des produits et des processus métier, tandis que le module IA réalise les traitements spécialisés liés à l'enrichissement et à la validation des données.

### 4.3.1 Principe de communication REST

REST (*Representational State Transfer*) [11] est un style architectural permettant à des applications distribuées de communiquer à travers le protocole HTTP. Dans cette approche, les fonctionnalités d'un système sont exposées sous forme de ressources accessibles via des URLs appelées points d'accès ou *endpoints*.

La communication repose sur des requêtes HTTP utilisant différentes méthodes selon l'opération souhaitée. Dans le cadre de ce projet, les échanges entre SAP Commerce Cloud et le module d'intelligence artificielle utilisent principalement la méthode `POST`. Cette méthode permet d'envoyer des informations vers un service distant afin de déclencher un traitement.

Une requête HTTP `POST` contient généralement :

- l'adresse de l'API appelée (*endpoint*) ;
- les informations nécessaires au traitement transmises dans le corps de la requête (généralement au format JSON [12]) ;
- les paramètres techniques permettant d'identifier le format des données échangées.

Dans cette architecture, SAP Commerce joue le rôle de client en envoyant les requêtes, tandis que le module d'intelligence artificielle agit comme un serveur capable de traiter ces demandes et de retourner les résultats correspondants.

### 4.3.2 API d’enrichissement et API de validation

Le module d’intelligence artificielle expose deux API REST principales consommées par SAP Commerce Cloud. Ces interfaces permettent d’intégrer les fonctionnalités d’enrichissement automatique et de contrôle qualité directement dans le cycle de gestion des produits.

#### API d’enrichissement

L’API d’enrichissement permet à SAP Commerce de transmettre les informations nécessaires à la génération automatique des contenus produit. Les données envoyées au module d’intelligence artificielle regroupent les informations descriptives et techniques nécessaires au traitement, telles que les informations générales du produit, sa marque, ses catégories, ses caractéristiques ainsi que les champs à enrichir et les langues cibles.

À partir de ces éléments, le module IA génère les contenus demandés et effectue les traitements associés, notamment la génération automatique des descriptions et leur adaptation aux différentes langues sélectionnées. Les résultats retournés sont ensuite exploités par SAP Commerce afin de mettre à jour les informations produit concernées.

#### API de validation

L’API de validation permet d’évaluer automatiquement la qualité des données produit existantes. Elle reçoit les informations nécessaires aux contrôles définis par le moteur de validation, notamment les données descriptives, les caractéristiques techniques et les éléments soumis aux différentes règles d’analyse.

Après traitement, cette API retourne les résultats de validation comprenant l’évaluation globale de la qualité des données, les anomalies détectées, leur niveau de sévérité ainsi que les corrections ou recommandations associées. Ces informations permettent ensuite à SAP Commerce d’assurer le suivi de la qualité des produits et d’appliquer les actions nécessaires.

### 4.3.3 Flux global de communication

Le fonctionnement global des échanges est illustré dans la figure 4.2. Lorsqu’une opération est déclenchée depuis SAP Commerce Cloud, la plateforme prépare les informations nécessaires puis transmet une requête vers l’API correspondante du module d’intelligence artificielle. Après exécution du traitement demandé, les résultats sont retournés à SAP Commerce afin d’être exploités dans le processus métier associé.

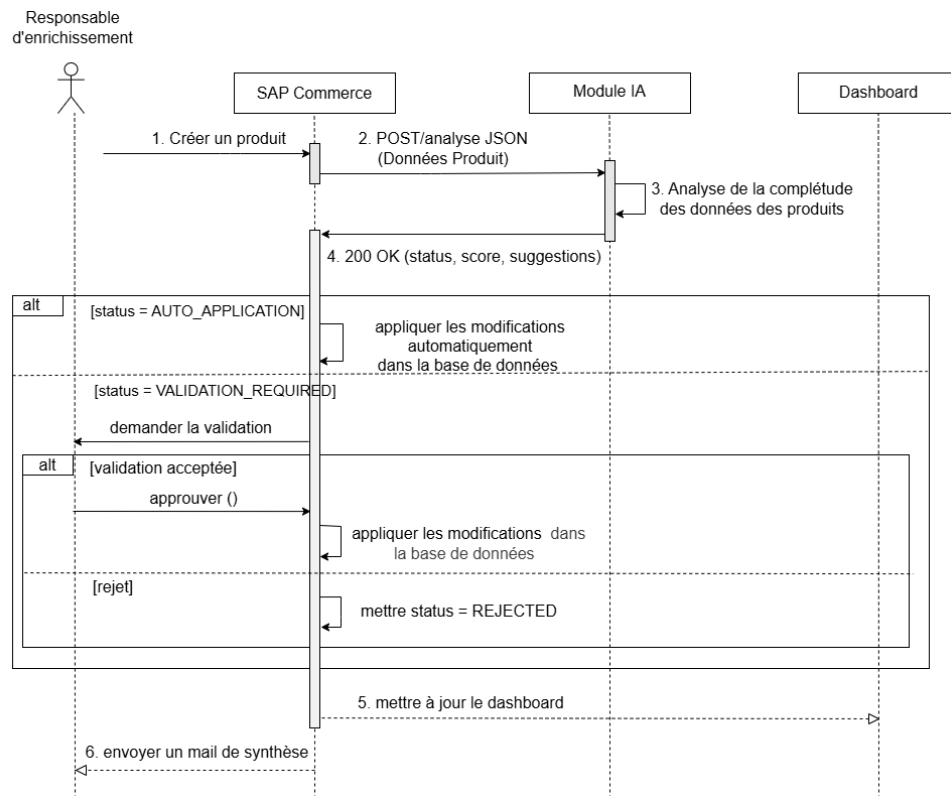


FIGURE 4.2 – Flux de communication entre SAP Commerce Cloud et le module IA

Cette organisation permet d'intégrer des capacités d'intelligence artificielle dans SAP Commerce Cloud tout en conservant une architecture modulaire, où chaque composant possède un rôle clairement défini.

## 4.4 Extension du modèle de données SAP Commerce

L'intégration du module d'intelligence artificielle nécessite l'adaptation du modèle de données existant de SAP Commerce afin de prendre en charge les nouvelles fonctionnalités d'enrichissement et de validation. Les extensions stockent les paramètres de traitement et les résultats générés par le module IA.

### 4.4.1 Adaptation du modèle produit

Le produit constitue l'élément central du processus d'enrichissement. Le modèle existant a donc été enrichi afin d'intégrer les informations nécessaires à la personnalisation des traitements d'intelligence artificielle.

Deux éléments de configuration ont été ajoutés au modèle produit. Le premier définit les informations du produit qui doivent faire l'objet d'un enrichissement ou d'une traduction. Le second précise les langues cibles dans lesquelles les contenus doivent être

générés.

Ces informations permettent d'adapter le traitement selon les besoins de chaque produit. Ainsi, tous les produits ne sont pas obligés de suivre le même processus d'enrichissement, ce qui offre davantage de flexibilité dans la gestion du catalogue.

#### **4.4.2 Gestion des résultats de validation**

La validation automatique génère plusieurs informations relatives à la qualité des données produit, notamment une évaluation globale, un état de validation ainsi que les problèmes identifiés.

Afin de conserver ces résultats et d'assurer leur suivi dans le temps, un modèle spécifique a été introduit pour représenter une opération de validation. Cette séparation évite le mélange des données propres au produit avec les informations générées par le processus de contrôle qualité.

Chaque validation effectuée correspond ainsi à un résultat indépendant associé au produit concerné. Cette approche conserve l'historique des différentes analyses réalisées et suit l'évolution de la qualité des données au cours du cycle de vie du produit.

#### **4.4.3 Gestion des anomalies détectées**

Lorsqu'une validation identifie des problèmes dans les informations produit, les anomalies détectées sont conservées séparément afin de faciliter leur consultation et leur traitement.

Chaque anomalie contient les informations nécessaires à son analyse, telles que le champ concerné, le niveau de gravité, la description du problème identifié ainsi que la suggestion de correction proposée.

Cette organisation structure les résultats de validation et fournit aux utilisateurs une vision claire des améliorations nécessaires.

#### **4.4.4 Relations entre les éléments du modèle**

Les différentes extensions réalisées forment un ensemble cohérent permettant de représenter le cycle complet de traitement. Un produit peut être soumis à plusieurs validations au cours de son cycle de vie et chaque validation peut générer plusieurs anomalies.

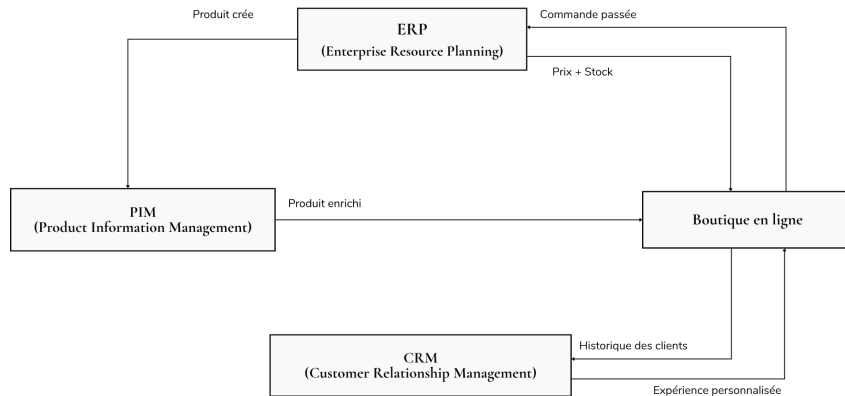


FIGURE 4.3 – Modèle de données pour la gestion de l’enrichissement et de la validation

Cette extension du modèle permet donc d’intégrer les capacités d’intelligence artificielle dans SAP Commerce tout en conservant une structure claire et évolutive, adaptée aux besoins d’enrichissement et de contrôle qualité des produits.

## 4.5 Interfaces Backoffice pour l’enrichissement et la validation

### 4.5.1 Vue d’ensemble du Backoffice SAP Commerce

Le Backoffice de SAP Commerce est l’interface d’administration centrale du système. C’est un espace sécurisé, accessible uniquement aux collaborateurs disposant des droits appropriés (administrateurs système, responsables de catalogues, validateurs de qualité, etc.). Contrairement à la boutique en ligne visible par les clients, le Backoffice reste un espace intérieur, pensé pour les workflows internes.

SAP Commerce fournit une interface Backoffice prête à l’emploi, appelée Backoffice Studio, dédiée à la gestion des produits, catégories, prix, stocks, commandes, etc. Cette interface repose sur le framework ZK [?], qui permet de construire des interfaces utilisateur riches en Java. Cependant, cette interface générique ne couvre pas tous les besoins métier spécifiques. Pour des processus métier particuliers, il est possible de développer des interfaces personnalisées.

Dans ce projet, deux interfaces métier ont été développées : l’une pour déclencher et suivre l’enrichissement de produits, l’autre pour consulter et agir sur les résultats de validation. Ces interfaces ont été développées en tant que composants Backoffice personnalisés, intégrés dans l’écosystème SAP Commerce.



### 4.5.2 Interface d'enrichissement

L'interface d'enrichissement permet aux responsables de catalogues de lancer l'enrichissement des descriptions des produits et d'en suivre l'exécution.

#### Éléments de l'interface

Lors du chargement de cette interface, l'utilisateur voit plusieurs éléments :

Une liste des produits sélectionnés pour enrichissement. Chaque produit est affiché avec son code, son nom, sa catégorie. L'utilisateur peut sélectionner un ou plusieurs produits via des checkboxes avant de lancer l'enrichissement. Cela permet d'enrichir des produits en masse, ce qui est beaucoup plus efficient que de traiter un produit à la fois.

Un formulaire de configuration avec deux sections principales. La première section demande à l'utilisateur quels champs enrichir. Par exemple, des checkboxes pour « description courte », « description longue », « spécifications ». L'utilisateur coche ceux qu'il souhaite enrichir. Ces informations seront stockées dans l'attribut `fieldsToTranslate` du `ProductModel`.

La deuxième section sert à sélectionner les langues cibles. Une liste déroulante ou un ensemble de checkboxes proposent les langues disponibles : français (FR), anglais (EN), espagnol (ES), allemand (DE), etc. L'utilisateur en sélectionne une ou plusieurs. Ces informations seront stockées dans l'attribut `targetLanguages` du `ProductModel`.

Un bouton « Lancer l'enrichissement » qui enclenche le processus. Quand on clique, les données du formulaire sont validées côté client (vérifier qu'au moins un champ et une langue sont sélectionnés), puis envoyées au serveur.

Une zone de statut et de progression qui affiche l'état de l'enrichissement. Avant le lancement, elle affiche « Prêt ». Après un clic sur le bouton, elle passe à « En cours ». Pendant l'enrichissement, la progression peut être affichée (par exemple, « 3 sur 10 produits traités »). Une fois terminé, elle affiche « Complété » et un résumé (nombre de produits enrichis, nombre d'erreurs).

#### Flux de sauvegarde des données

Quand l'utilisateur clique sur « Lancer l'enrichissement », plusieurs opérations se déroulent en arrière-plan.

Premièrement, une Facade reçoit l'appel du Backoffice. Elle valide les paramètres : les produits sélectionnés existent-ils ? L'utilisateur a-t-il le droit d'enrichir ces produits ? Les langues sélectionnées sont-elles valides ?

Deuxièmement, la Facade met à jour les `ProductModels` concernés, en sauvegardant

la configuration : `fieldsToTranslate` reçoit la liste des champs sélectionnés, `targetLanguages` reçoit la liste des langues. Cette sauvegarde est cruciale car elle trace quelle configuration a été utilisée pour chaque produit. Si un utilisateur peut modifier la configuration après avoir lancé l'enrichissement, la traçabilité serait perdue.

Troisièmement, la Facade déclenche un processus asynchrone (un job, un workflow, ou un appel direct au service métier) qui appelle le microservice pour enrichir chaque produit. Cet asynchrone est important : l'interface Backoffice retourne immédiatement du contrôle à l'utilisateur sans attendre la fin de l'enrichissement (qui peut prendre plusieurs secondes ou minutes si le microservice est lent).

### Affichage des résultats

Une fois l'enrichissement terminé, l'interface affiche les résultats. Pour chaque produit, on voit :

La description originale (par exemple, en français).

Les descriptions enrichies par le microservice, organisées par langue. Par exemple, en anglais : « High-performance running shoe designed for comfort and durability », en espagnol : « Zapato de running de alto rendimiento diseñado... », en allemand : « Hochleistungs-Laufschuh... »

L'utilisateur peut comparer la description originale avec les versions enrichies et juger de leur qualité avant validation. Si une description ne lui plaît pas, il peut la modifier manuellement (si cette fonctionnalité est implémentée) ou l'enrichissement de ce champ peut être relancé ultérieurement.



FIGURE 4.4 – Aperçu de l'interface d'enrichissement dans le Backoffice

### 4.5.3 Interface de validation

L'interface de validation permet aux validateurs de qualité de consulter les résultats de validation des produits, d'identifier les anomalies et de prendre les mesures correctives appropriées.

## Affichage des résultats de validation

Quand on ouvre l'interface de validation pour un produit, on voit d'abord un score de qualité global, affiché de manière saillante. Par exemple, « Score de qualité : 85/100 ». Ce score à lui seul donne une impression rapide de la qualité du produit.

Un statut synthétique accompagne le score. Par exemple : « Produit valide » (si le score est  $\geq 90$ ), « Produit valide avec mise en garde » (si  $70 \leq \text{score} < 90$ ), « Produit à réviser » (si  $\text{score} < 70$ ). Le statut est souvent codé par couleur : vert pour valide, orange pour mise en garde, rouge pour à réviser.

## Liste des anomalies détectées

Sous le score, on trouve la liste des anomalies détectées lors de la validation. Chaque anomalie est affichée avec :

Le champ concerné (par exemple, « description\_en »).

Le niveau de sévérité (INFO, AVERTISSEMENT, ERREUR). Une anomalie de sévérité ERROR bloque possiblement certaines actions (publication du produit) ; une AVERTISSEMENT est signalée mais moins critique ; une INFO est informative.

Une description du problème en termes lisibles (par exemple, « La description en anglais est inférieure à 50 caractères »).

Une suggestion de correction (par exemple, « Ajouter au moins 30 caractères pour une description complète »).

## Actions correctives

Pour chaque anomalie, l'interface propose une action. Dans les cas simples, il est possible d'accepter une suggestion de correction automatique, ce qui applique la modification au produit immédiatement. Par exemple, si le microservice suggère « Ajouter un point à la fin de la description », un clic sur « Appliquer la correction » effectue cette modification.

Pour les cas plus complexes, l'utilisateur peut éditer manuellement le champ concerné via un éditeur de texte intégré à l'interface.

## Workflow de validation

Le processus complet ressemble à ceci. L'utilisateur ouvre l'interface de validation et cherche un produit qui nécessite validation. Il sélectionne le produit. Le système récupère le ProductValidationResult le plus récent associé à ce produit et affiche le score, le statut et les anomalies.

L'utilisateur lit les anomalies et décide d'agir. Si les anomalies sont mineures et que les suggestions automatiques conviennent, il clique sur « Appliquer toutes les corrections ». Le système applique les corrections et marque le produit comme « Validé ».

Si l'utilisateur n'est pas d'accord avec une suggestion, il peut éditer le champ concerné manuellement, puis relancer la validation pour voir si les corrections éliminent l'anomalie.

Si un score est critique ( $< 50$ , par exemple), l'interface peut empêcher certaines actions comme la publication du produit jusqu'à correction.



FIGURE 4.5 – Aperçu de l'interface de validation dans le Backoffice

Les deux interfaces d'enrichissement et de validation permettent à l'utilisateur de piloter manuellement les processus. Cependant, pour une véritable automatisation et pour éviter l'intervention humaine répétitive, SAP Commerce dispose d'un moteur de workflows. C'est le sujet de la section suivante.

## 4.6 Workflows métier et automatisation

### 4.6.1 Workflows dans SAP Commerce

Un workflow est une séquence d'étapes automatisées qui exécutent un processus métier selon une logique prédéfinie. Dans le contexte de SAP Commerce, le moteur de workflow gère l'ordonnancement des étapes, les transitions conditionnelles, l'appel des actions métier et l'enregistrement de l'historique.

Pourquoi utiliser un workflow au lieu d'écrire simplement une série d'appels de services ? Plusieurs avantages le justifient. D'abord, un workflow rend le processus visible et compréhensible pour les non-développeurs : un responsable métier peut lire la définition du workflow et comprendre comment fonctionnent les processus. Ensuite, un workflow est flexible : modifier le processus ne nécessite pas de redéployer du code Java ; on modifie la configuration du workflow. Troisièmement, les workflows offrent des capacités avancées : parallélisation d'étapes, gestion des erreurs, relance automatique, etc. Enfin, les workflows fournissent une traçabilité complète : chaque étape est loggée, et on peut consulter l'historique d'exécution.

SAP Commerce utilise un moteur de workflow déclaratif. Les workflows ne sont pas écrits en Java, mais définis via un format de configuration (XML ou une interface graphique appelée Workflow Builder). On décrit : « L'étape A s'exécute, puis on teste une condition, si elle est vraie on va à l'étape B, sinon à l'étape C ». Le moteur lit cette configuration et l'exécute.

#### Implémentation dans le projet

Dans ce projet, les workflows ont été définis via le mécanisme d'impex de SAP Commerce. Impex est un format texte de configuration qui permet de définir des données, des configurations, et des workflows. Les définitions de workflow ont été rédigées en impex et importées dans SAP Commerce lors du déploiement.

Chaque étape du workflow fait référence à une action métier. Une action métier est une classe Java implémentant une interface standard (par exemple, `WorkflowAction`). Quand le workflow atteint cette étape, il instancie la classe et exécute sa méthode `execute()`.

## 4.6.2 Workflow d'enrichissement

Le workflow d'enrichissement automatise complètement le processus d'enrichissement d'un produit, du déclenchement jusqu'à la persistance des résultats.

### Étapes du workflow

Le workflow se déroule selon une séquence bien définie :

**Étape 1 : Déclenchement.** Le workflow est déclenché quand l'utilisateur clique sur « Lancer l'enrichissement » dans l'interface Backoffice. À ce moment, les `ProductModels` concernés ont déjà été mis à jour avec la configuration (champs à enrichir, langues cibles). Le workflow reçoit en paramètre la liste des IDs de produits à enrichir.

**Étape 2 : Préparation des données.** Une action métier appelée `ProductEnrichmentAutomatedAction` s'exécute. Elle récupère le `ProductModel` depuis la base de données, extrait les informations pertinentes (code, nom, descriptions actuelles, champs à traduire, langues cibles) et construit un objet `ProductEnrichmentRequest` prêt à envoyer au microservice.

**Étape 3 : Appel au microservice.** Une seconde action (ou la même, selon l'implémentation) utilise `RestTemplate` pour envoyer la requête au microservice et récupère la réponse JSON, désérialisée en objet `ProductEnrichmentResponse` par Jackson.

**Étape 4 : Traitement de la réponse.** L'action vérifie que la réponse indique un succès (`success = true`). Si oui, elle procède à l'étape suivante. Si non (par exemple, si le microservice a signalé une erreur métier), elle redirige le workflow vers une étape de gestion d'erreur.

**Étape 5 : Mise à jour du produit.** L'action extrait les descriptions enrichies de la réponse (descriptions par langue) et les sauvegarde dans le `ProductModel`. Concrètement, cela signifie mettre à jour les attributs `description_en`, `description_es`, etc. avec les valeurs retournées par le microservice. Ces modifications sont persistées en base de données via le `ModelService`.

**Étape 6 : Déclenchement du workflow de validation.** Après enrichissement réussi, le workflow d'enrichissement déclenche automatiquement le workflow de validation pour le même produit (voir section 4.6.3). Cela crée une chaîne d'automatisation : enrichir puis valider immédiatement.

**Étape 7 : Fin du workflow.** Une étape de conclusion enregistre que le workflow s'est terminé avec succès, log un message et éventuellement notifie l'utilisateur que l'enrichissement est terminé (voir section 4.8).

## Gestion des erreurs

À plusieurs points du workflow, des erreurs peuvent survenir. Le microservice peut être inaccessible (serveur web down, timeout réseau). La réponse du microservice peut être malformée (JSON invalide, champs manquants). La base de données peut être inaccessible lors de la sauvegarde.

Pour chacune de ces situations, le workflow contient une branche de gestion d'erreur. Typiquement, si un appel au microservice échoue, le workflow saute à une étape « Gestion d'erreur » qui :

- Enregistre l'erreur dans les logs du système.

- Notifie l'administrateur système qu'un enrichissement a échoué et a besoin d'intervention.

- Marque le produit avec un statut « Enrichissement échoué - Nouvelle tentative requise ».

- Optionnellement, déclenche une tentative automatique après un délai (par exemple, réessayer après 5 minutes).

Sans cette gestion robuste des erreurs, un problème réseau passager bloquerait indéfiniment l'enrichissement.



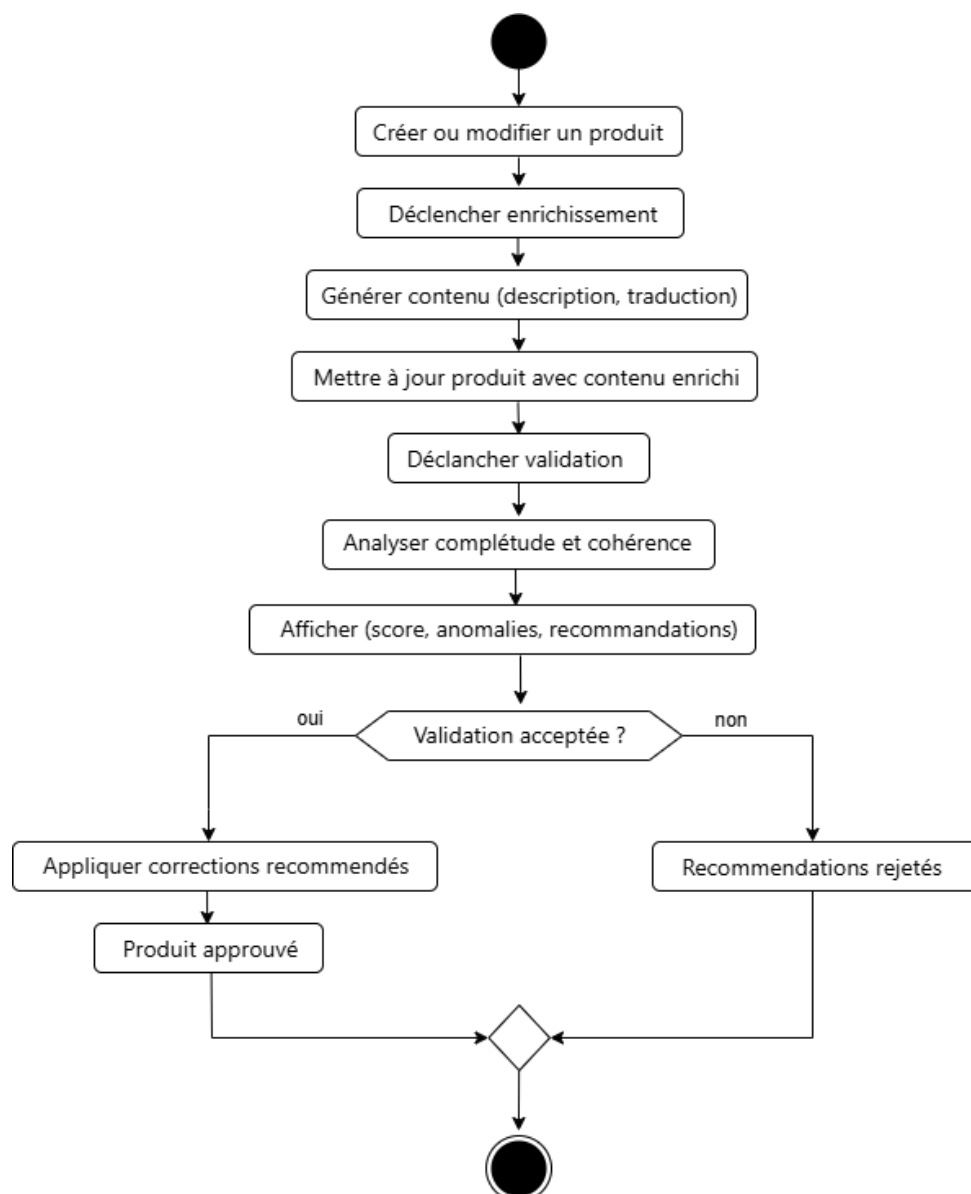


FIGURE 4.6 – Diagramme d'activités du workflow d'enrichissement

### 4.6.3 Workflow de validation

Le workflow de validation fonctionne de manière similaire au workflow d'enrichissement, avec des différences métier importantes.

#### Étapes du workflow

**Étape 1 : Déclenchement.** Le workflow de validation est déclenché soit manuellement par l'utilisateur (via l'interface de validation), soit automatiquement à la fin du workflow d'enrichissement. Il reçoit en paramètre l'ID du produit à valider.

**Étape 2 : Appel au microservice de validation.** Une action appelle l'endpoint de validation du microservice avec un objet `ProductValidationRequest` contenant les

descriptions actuelles du produit (en toutes les langues) et d'autres métadonnées.

**Étape 3 : Réception du score et des anomalies.** Le microservice retourne une réponse contenant un score de qualité (0-100), un statut et une liste d'anomalies détectées.

**Étape 4 : Persistance des résultats.** Une action crée un nouvel enregistrement `ProductValidationResult` avec le score et le statut reçus. Pour chaque anomalie retournée par le microservice, une instance de `ValidationAnomaly` est créée et liée au `ProductValidationResult`. Ces entités sont sauvegardées en base de données.

**Étape 5 : Prise de décision basée sur le score.** Le workflow teste le score. Si le score est  $\geq 80$  (par exemple), le produit est marqué comme « Validé ». Si  $60 \leq \text{score} < 80$ , le statut devient « Avertissement ». Si  $\text{score} < 60$ , le statut devient « À réviser ». Ce seuil peut être configuré.

**Étape 6 : Notification.** Un email est généré et envoyé au responsable de la validation (voir section 4.8) avec un résumé : score, statut, anomalies critiques. Si le statut est « À réviser », un email additionnel peut être envoyé au responsable produit pour action.

**Étape 7 : Fin du workflow.** Enregistrement de la completion du workflow.

## Différences avec le workflow d'enrichissement

Contrairement au workflow d'enrichissement qui modifie le `ProductModel`, le workflow de validation ne modifie pas les descriptions : il les analyse seulement. Le workflow d'enrichissement envoie des données au microservice et en récupère d'autres ; le workflow de validation envoie des données et reçoit une évaluation, pas des données de remplacement.

De plus, le workflow de validation génère toujours une notification, tandis que le workflow d'enrichissement peut la générer optionnellement.

Maintenant que les workflows orchestrent l'automation, il est instructif de voir comment tout s'imbrique. La section suivante trace le flux complet d'une requête utilisateur à travers l'ensemble du système.

## 4.7 Flux complet d'exécution et orchestration

### 4.7.1 Scénario concret : enrichissement d'un produit

Pour bien comprendre comment tous les éléments décrits précédemment s'articulent, traçons le parcours complet d'une requête utilisateur.

Une responsable de catalogue, nommée Marie, travaille dans le Backoffice. Elle gère un produit phare de l'entreprise, un article de chaussure sportive avec le code

« CHAUSSURE-001 » et le nom « Chaussure de sport Running ». Ce produit a une description courte en français : « Chaussure de sport de haute qualité ». Marie souhaite enrichir ce produit en générant des descriptions de qualité professionnelle en anglais, espagnol et allemand.

Elle navigue jusqu'à l'interface d'enrichissement. Elle sélectionne le produit « CHAUSSURE-001 ». Elle coche le champ « Description courte » pour enrichissement. Elle sélectionne les langues « Anglais », « Espagnol », « Allemand ». Elle clique sur le bouton « Lancer l'enrichissement ».

## 4.7.2 Étapes techniques du flux complet

### Étape 1 : Validation et préparation (Couche Présentation/Facade)

L'interface Backoffice envoie une requête HTTP vers le serveur SAP Commerce, transmettant les paramètres : `productCode` = « CHAUSSURE-001 », `fieldsToTranslate` = [« description »], `targetLanguages` = [« en », « es », « de »].

Une Facade appelée `ProductEnrichmentFacade` reçoit cette requête. Elle effectue des validations préliminaires :

Existe-t-il un produit avec le code CHAUSSURE-001 ? (Oui)

L'utilisateur actuel (Marie) a-t-elle les droits pour enrichir les produits ? (Oui)

Les langues « en », « es », « de » sont-elles des codes valides ? (Oui)

Si une de ces validations échoue, la Facade retourne immédiatement une erreur à l'interface, qui l'affiche à l'utilisateur.

### Étape 2 : Sauvegarde de la configuration

Après validation, la Facade met à jour le `ProductModel` de CHAUSSURE-001 en base de données :

```
fieldsToTranslate = [« description »]
```

```
targetLanguages = [« en », « es », « de »]
```

Cette sauvegarde est persistée via le `ModelService`, qui utilise un DAO interne pour mettre à jour la base de données.

### Étape 3 : Déclenchement du workflow d'enrichissement

Après cette sauvegarde, la Facade déclenche le workflow d'enrichissement, en lui passant l'ID du produit CHAUSSURE-001.

## Étape 4 : Orchestration métier (Couche Service)

Le workflow d'enrichissement s'exécute. La première action métier (`ProductEnrichmentAutomatedAction`) s'exécute. Elle :

Récupère le `ProductModel` `CHAUSSURE-001` de la base de données.

Extrait les informations :

- `productCode` = « CHAUSSURE-001 »
- `productName` = « Chaussure de sport Running »
- `description` = « Chaussure de sport de haute qualité »
- `fieldsToTranslate` = [« description »]
- `targetLanguages` = [« en », « es », « de »]

Construit un objet DTO de type `ProductEnrichmentRequest` avec ces données.

## Étape 5 : Sérialisation et communication HTTP

L'action appelle un service métier `ProductEnrichmentService`, en lui transmettant le `ProductModel`.

Le service utilise `RestTemplate` [?] pour appeler le microservice. `RestTemplate` :

Prend l'objet `ProductEnrichmentRequest` et utilise la bibliothèque `Jackson` [?] pour le sérialiser en JSON :

```
{
  "productCode": "CHAUSSURE-001",
  "productName": "Chaussure de sport Running",
  "description": "Chaussure de sport de haute qualité",
  "fieldsToTranslate": ["description"],
  "targetLanguages": ["en", "es", "de"]
}
```

Construit une requête HTTP POST vers l'URL du microservice (par exemple, `http://microservice-ia:8000/api/v1/enrich`).

Ajoute les en-têtes HTTP appropriés : `Content-Type: application/json`, `Authorization: Bearer <token>`, etc.

Établit une connexion TCP vers le serveur microservice.

Envoie la requête avec le JSON en corps.

Attend la réponse pendant au maximum 30 secondes (timeout configuré).

## Étape 6 : Traitement du microservice

Le microservice Python reçoit la requête. Il exécute le traitement décrit au chapitre 3 : génération de descriptions enrichies en utilisant les modèles de langage, traduction en les trois langues cibles. Après environ 2-3 secondes, il construit une réponse JSON :

```
{
  "success": true,
  "enrichedData": {
    "en": "High-performance sports shoe combining comfort, durability,
          and style. Perfect for everyday running and athletic activities.",
    "es": "Zapato deportivo de alto rendimiento que combina comodidad,
          durabilidad y estilo. Perfecto para correr diariamente...",
    "de": "Hochleistungs-Sportschuh, der Komfort, Haltbarkeit und Stil
          kombiniert. Perfekt für alltägliches Laufen..."
  },
  "timestamp": "2024-01-15T14:30:45Z",
  "processingTimeMs": 2850
}
```

Il envoie cette réponse en tant que corps d'une réponse HTTP 200 OK.

## Étape 7 : Désérialisation de la réponse

RestTemplate reçoit la réponse HTTP. Jackson désérialise le JSON en un objet Java de type `ProductEnrichmentResponse` :

```
ProductEnrichmentResponse response = new ProductEnrichmentResponse();
response.setSuccess(true);
Map<String, String> enrichedData = new HashMap<>();
enrichedData.put("en", "High-performance sports shoe...");
enrichedData.put("es", "Zapato deportivo...");
enrichedData.put("de", "Hochleistungs-Sportschuh...");
response.setEnrichedData(enrichedData);
response.setTimestamp(LocalDateTime.parse("2024-01-15T14:30:45Z"));
```

Le service métier reçoit cet objet Java structuré.

### Étape 8 : Mise à jour du ProductModel

Le service métier extrait les descriptions enrichies de la réponse. Il met à jour le ProductModel CHAUSSURE-001 :

description\_en = « High-performance sports shoe... »

description\_es = « Zapato deportivo... »

description\_de = « Hochleistungs-Sportschuh... »

Le ModelService persiste ces modifications en base de données via les DAOs.

### Étape 9 : Déclenchement du workflow de validation

À la fin du workflow d'enrichissement, le workflow de validation est déclenché automatiquement pour le même produit CHAUSSURE-001.

### Étape 10 : Validation des descriptions

Le workflow de validation appelle le microservice avec l'endpoint de validation, transmettant les descriptions enrichies. Le microservice analyse chaque description selon ses critères de qualité (longueur, richesse du vocabulaire, cohérence, complétude) et retourne :

```
{
  "score": 87,
  "status": "VALID_WITH_WARNINGS",
  "anomalies": [
    {
      "field": "description_es",
      "severity": "WARNING",
      "message": "Description espagnole un peu courte (65 caractères)",
      "suggestion": "Ajouter des détails sur les matériaux ou les performances"
    }
  ],
  "timestamp": "2024-01-15T14:31:00Z"
}
```

### Étape 11 : Persistance des résultats de validation

Le workflow de validation reçoit cette réponse. Il crée un nouvel enregistrement ProductValidationResult :

ID = 42 (généralisé automatiquement)

Product = CHAUSSURE-001

Score = 87

Status = VALID\_WITH\_WARNINGS

ValidationDate = 2024-01-15 14 :31 :00

Il crée aussi une entité ValidationAnomaly :

ID = 101

ProductValidationResult = 42

Field = description\_es

Severity = WARNING

Message = « Description espagnole un peu courte (65 caractères) »

Suggestion = « Ajouter des détails sur les matériaux ou les performances »

Ces deux entités sont sauvegardées en base de données.

## Étape 12 : Notification par email

Après validation complète, le workflow déclenche l'envoi d'un email (voir section 4.8).  
Un template Velocity génère le contenu :

Objet : Enrichissement et validation du produit CHAUSSURE-001

Bonjour,

Le produit "Chaussure de sport Running" (CHAUSSURE-001) a été enrichi et validé avec succès.

Score de qualité : 87/100

Statut : Valide avec mise en garde

Anomalies détectées :

- description\_es (AVERTISSEMENT) : Description espagnole un peu courte.

Suggestion : Ajouter des détails sur les matériaux ou les performances.

Vous pouvez consulter les détails dans l'interface de validation.

Cordialement,

Système d'enrichissement de produits

Cet email est envoyé à la responsable de validation (ou à Marie si elle l'a demandé).

### Étape 13 : Retour vers l'interface

L'interface Backoffice, qui attendait le résultat de la Facade (depuis l'étape 1), reçoit enfin une réponse. L'opération asynchrone en arrière-plan s'est terminée et le résultat remonte vers l'interface.

L'interface affiche :

« Enrichissement terminé avec succès »

Les descriptions enrichies en trois langues

Le score de validation (87/100)

Les anomalies détectées (avec possibilité d'appliquer les corrections suggérées)

Marie peut maintenant consulter les descriptions enrichies. Elle voit que la description en espagnol est un peu courte (selon l'avis du microservice), mais dans l'ensemble le résultat est satisfaisant. Elle clique sur « Approuver et publier » pour marquer le produit comme validé et le rendre visible sur la boutique en ligne.

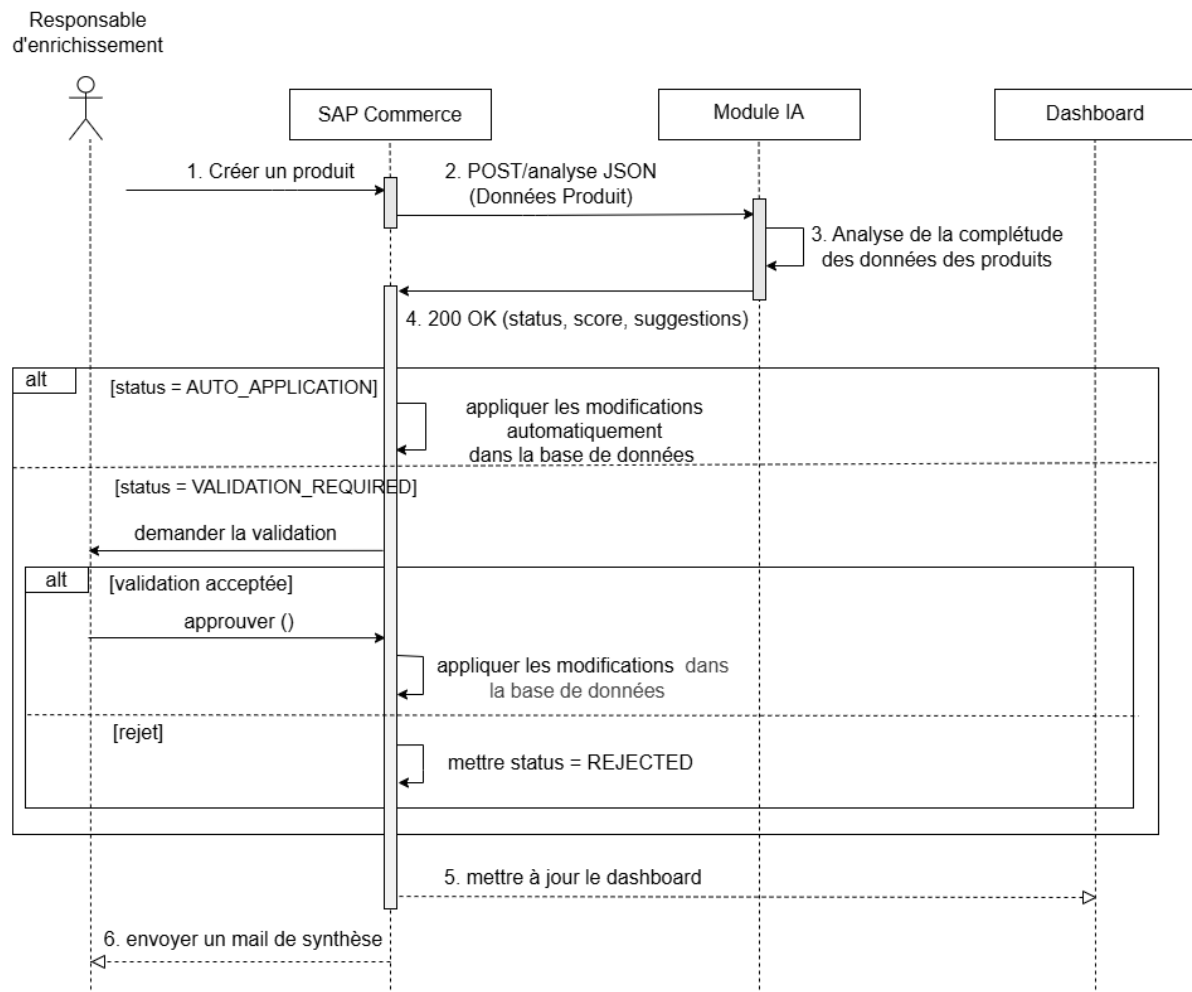


FIGURE 4.7 – Diagramme de séquence global de l'interaction utilisateur et des processus système

Cet enchaînement illustre l'interopérabilité entre les composants du système :



l'interface, les Facades, les services métier, la sérialisation des données, la communication HTTP, le traitement du microservice, la persistance, les workflows et les notifications. Aucun de ces éléments ne fonctionne de manière isolée ; ils forment un ensemble cohérent où chaque maillon est indispensable.

## 4.8 Système de notification par courrier électronique

### 4.8.1 Moteur d'email dans SAP Commerce

SAP Commerce intègre un moteur d'email complet qui gère l'envoi de courriers électroniques à partir du système. Ce moteur découple la génération du contenu de l'email de son envoi via SMTP, ce qui offre plusieurs avantages en termes de performance et de fiabilité.

La logique est la suivante : quand un événement métier survient (fin d'enrichissement, fin de validation, erreur), un processus génère le contenu de l'email (le corps du message, le sujet, les destinataires). Cet email est ajouté à une queue (une file d'attente). Un worker en arrière-plan consomme cette queue et envoie les emails via le serveur SMTP configuré.

Cette séparation présente plusieurs avantages. D'abord, la génération du contenu peut être lente (si elle implique des requêtes en base de données, des appels externes, etc.), mais cela n'impacte pas l'interface utilisateur : l'interface reçoit le contrôle immédiatement, tandis que la génération du contenu s'effectue en arrière-plan. Ensuite, si le serveur SMTP est temporairement inaccessible, les emails restent dans la queue et sont renvoyés ultérieurement une fois que le serveur SMTP redevient disponible. Aucun email n'est perdu.

### 4.8.2 Moteurs de template Velocity

Plutôt que de générer les emails via des concaténations de chaînes de caractères en Java (ce qui serait verbeux et peu flexible), SAP Commerce utilise le moteur **Velocity** [?].

**Velocity** [?] est un moteur de template simple mais puissant. Un template Velocity est un fichier texte contenant du texte fixe et des directives de template délimitées par des symboles spéciaux (généralement \$ pour les variables, # pour les directives). Par exemple :

Objet : Enrichissement du produit \$productName

Bonjour,

Le produit "\$productName" (code : \$productCode) a été enrichi avec succès.

Descriptions générées :

```
#foreach($language in $targetLanguages)
  - $language : ${enrichedDescriptions.get($language)}
#end
```

Score de validation : \$validationScore/100

```
#if($validationScore >= 80)
```

Produit validé. Prêt pour publication.

```
#else
```

Avertissement : le produit nécessite une révision avant publication.

```
#end
```

Anomalies détectées :

```
#foreach($anomaly in $anomalies)
  - [$anomaly.field] ($anomaly.severity) : $anomaly.message
#end
```

Cordialement,

L'équipe de gestion des produits

Quand on fournit des données à ce template (productName = « Chaussure de sport Running », productCode = « CHAUSSURE-001 », targetLanguages = [« en », « es », « de »], validationScore = 87, etc.), Velocity remplace les variables par leurs valeurs, évalue les directives conditionnelles (`#if`), itère sur les listes (`#foreach`), et génère le contenu final.

Pourquoi utiliser Velocity plutôt que de coder l'email directement en Java ? Plusieurs raisons. D'abord, les templates Velocity sont écrits en texte brut, facilement éditables. Un responsable métier peut modifier un template sans compétences Java. Ensuite, modifier un template ne nécessite pas une recompilation et un redéploiement de l'application : SAP Commerce peut recharger les templates dynamiquement. Enfin, les templates permettent une séparation nette entre la présentation (ce que dit l'email) et la logique (ce qui construit les données).

### 4.8.3 Configuration SMTP et Mailtrap

SMTP (Simple Mail Transfer Protocol) est le protocole standard pour envoyer des emails. Pour envoyer un email, SAP Commerce a besoin de la configuration d'un serveur SMTP : l'adresse du serveur, le port, les identifiants (login et mot de passe), etc.

En production, le serveur SMTP est généralement un serveur mail de l'entreprise (Google Workspace, Microsoft Exchange, ou un serveur mail auto-hébergé). Cependant, en environnement de développement et de test, envoyer de vrais emails est problématique : on risque d'envoyer des emails de test à des adresses email réelles d'utilisateurs.

Pour résoudre ce problème, des services comme Mailtrap existent. Mailtrap simule un serveur SMTP mais, au lieu de réellement envoyer les emails, les capture et les stocke pour inspection. Cela permet aux développeurs de tester le fonctionnement du moteur d'email sans risque d'envoyer des emails indésirables.

Dans ce projet, Mailtrap a été utilisé pour les environnements de développement et de test. La configuration SAP Commerce pointait vers le serveur Mailtrap. Lors du lancement d'un enrichissement ou d'une validation en développement, les emails générés étaient capturés par Mailtrap et visibles via son interface web. En production, la configuration serait changée pour pointer vers le vrai serveur SMTP de l'entreprise.

### 4.8.4 Processus complet de notification

#### Déclenchement

Quand le workflow de validation se termine, une action métier appelle le moteur d'email de SAP Commerce. Elle spécifie :

- Le template à utiliser (par exemple, `enrichmentAndValidationNotification`)

- Les données à passer au template (ProductModel, ProductValidationResult, ValidationAnomalies, etc.)

- Le destinataire de l'email (par exemple, l'adresse email de la responsable de validation)

- Le sujet de l'email (optionnel si le template le définit)

#### Génération du contenu

SAP Commerce charge le template spécifié (du fichier système ou de la base de données). Il exécute le moteur Velocity avec les données fournies. Velocity remplace les variables par leurs valeurs et évalue les directives.

Par exemple, si le template contient :

```
#foreach($anomaly in $anomalies)
```

```
- $anomaly.field : $anomaly.message  
#end
```

Et que `$anomalies` contient deux anomalies, Velocity génère :

- `description_es` : Description espagnole un peu courte
- `description_en` : Point manquant à la fin

Le contenu final généré ressemble à un email normal, avec un sujet, un corps, et éventuellement des pièces jointes (PDF, images, etc.).

### Ajout à la queue et envoi

L'email généré est créé en tant qu'objet (sujet, corps, destinataires, pièces jointes) et ajouté à une queue de traitement. Un worker (processus en arrière-plan ou job batch) consomme cette queue périodiquement.

Pour chaque email dans la queue, le worker utilise la configuration SMTP pour ouvrir une connexion au serveur SMTP, authentifie SAP Commerce, et envoie l'email. Si l'envoi échoue (serveur inaccessible, destinataire invalide), le worker enregistre l'erreur et peut relancer l'envoi ultérieurement.

### Cas d'usage dans le projet

Trois types de notifications ont été implémentées :

**Email après enrichissement réussi :** « Votre enrichissement de produit est terminé. 3 produits enrichis avec succès. 0 erreurs. »

**Email après validation complète :** « Validation terminée. Produit CHAUSSURE-001 : Score 87/100. 1 avertissement détecté. Voir les détails dans le Backoffice. »

**Email d'alerte en cas d'erreur :** « Enrichissement du produit CHAUSSURE-001 a échoué. Le microservice n'a pas répondu dans le délai imparti. Veuillez réessayer ou contacter l'administrateur système. »

## 4.9 Conclusion du chapitre

L'intégration dépasse le simple appel HTTP pour embrasser une dimension systémique via six axes complémentaires.

Ce chapitre a présenté ces six axes d'intégration :

**[Figure 4.9 : Diagramme de flux - Système de notifications]**

Insérer un diagramme montrant :

- Workflow ou Service métier
- Appel au moteur Email avec template et données
- Moteur Velocity génère contenu
- Email créé (sujet, corps, destinataires)
- Ajout à la queue d'emails
- Worker consomme queue
- Configuration SMTP
- Connexion au serveur SMTP
- Envoi effectif de l'email
- Utilisateur reçoit email
- Feedback de succès/erreur

Premièrement, l'architecture logicielle de SAP Commerce elle-même, organisée en couches (Présentation, Facade, Service, DAO, Modèle), où chaque couche a une responsabilité bien définie. Les développements du projet se situent dans toutes les couches.

Deuxièmement, la communication technique entre SAP Commerce et le microservice via REST, RestTemplate et Jackson. Cette communication est robuste, gère les erreurs, et persiste les résultats.

Troisièmement, l'extension du modèle de données pour accueillir les nouvelles entités métier : l'enrichissement de ProductModel avec des attributs de configuration, et la création de ProductValidationResult et ValidationAnomaly pour persister les résultats de validation avec historique.

Quatrièmement, les interfaces Backoffice personnalisées qui permettent aux utilisateurs métier de déclencher l'enrichissement et de consulter les résultats de validation. Ces interfaces rendent accessibles les capacités du microservice sans nécessiter de connaissances techniques.

Cinquièmement, l'automatisation via les workflows métier. Les workflows d'enrichissement et de validation orchestrent les étapes, gèrent les erreurs, et assurent la traçabilité. Ils rendent le processus visible et modifiable sans intervention de développeurs.

Sixièmement, le système de notifications par email, utilisant le moteur Email de SAP Commerce, les templates Velocity, et la configuration SMTP. Les notifications informent les utilisateurs en temps réel de la progression et des anomalies.

Au-delà de ces six axes, ce qui rend cette intégration complète c'est leur cohésion. Aucun de ces éléments ne fonctionne de manière isolée. L'interface déclenche un workflow,

qui appelle un service, qui appelle le microservice via RestTemplate, qui est désérialisé par Jackson, qui met à jour le modèle via un DAO, qui persiste les résultats, qui sont ensuite affichés par l'interface, et dont on notifie l'utilisateur par email. C'est cette orchestration globale qui constitue une véritable intégration.

Le chapitre suivant aborde la validation de cette intégration : comment s'assurer que le système fonctionne correctement, que les enrichissements sont de qualité, que les erreurs sont gérées robustement, et que les performances sont acceptables. Cela passe par les stratégies de test, les expérimentations, et l'analyse des résultats.

# Conclusion et perspectives

Récapitulation du travail réalisé

Enrichissements possibles.

**Annexe A**

**Annexe 1**



# Bibliographie

# Netographie

- [1] DECADE. Conseil et ingénierie e-commerce depuis 1988. <https://www.decade.fr/agence/>. Consulté le 01/06/2026.
- [2] ETANCO. Fabricant de fixations et systèmes pour l'enveloppe du bâtiment. <https://www.etanco.fr/>. Consulté le 01/06/2026.
- [3] IBM. What is e-commerce ? <https://www.ibm.com/topics/ecommerce>. Consulté le 01/06/2026.
- [4] SAP. Qu'est-ce qu'un erp (entreprise resource planning) ? <https://www.sap.com/france/products/erp/what-is-erp.html>. Consulté le 01/06/2026.
- [5] Akeneo. Qu'est-ce qu'un pim (product information management) ? <https://www.akeneo.com/fr/qu-est-ce-qu-un-pim/>. Consulté le 01/06/2026.
- [6] SAP. Sap commerce cloud - documentation officielle. [https://help.sap.com/docs/SAP\\_COMMERCE\\_CLOUD](https://help.sap.com/docs/SAP_COMMERCE_CLOUD). Consulté le 01/06/2026.
- [7] Salesforce. Qu'est-ce que le crm ? <https://www.salesforce.com/fr/crm/definition-crm/>. Consulté le 01/06/2026.
- [8] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/scrum-guide.html>. Consulté le 01/06/2026.
- [9] Unified modeling language (uml) specification. <https://www.omg.org/spec/UML/>. Consulté le 01/06/2026.
- [10] Microservices - martin fowler. <https://martinfowler.com/articles/microservices.html/>. Consulté le 01/06/2026.
- [11] Architectural styles and the design of network-based software architectures (rest). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm/>. Consulté le 01/06/2026.
- [12] Json (javascript object notation). <https://www.json.org/json-en.html/>. Consulté le 01/06/2026.
- [13] Fastapi documentation. <https://fastapi.tiangolo.com/>. Consulté le 01/06/2026.

- [14] Python software foundation. <https://www.python.org/psf-landing/>. Consulté le 01/06/2026.